



POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



Memory and Cache

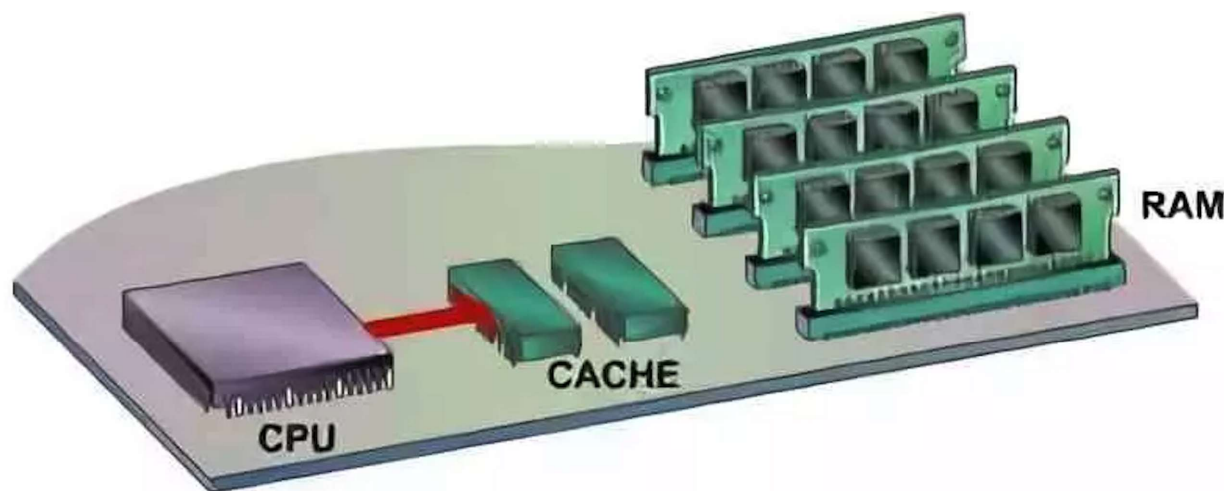
Faculty: Computer science – IT1

Instructor: Viet Hoang Pham

Email: vietph@ptit.edu.vn

Phone: 0945296388

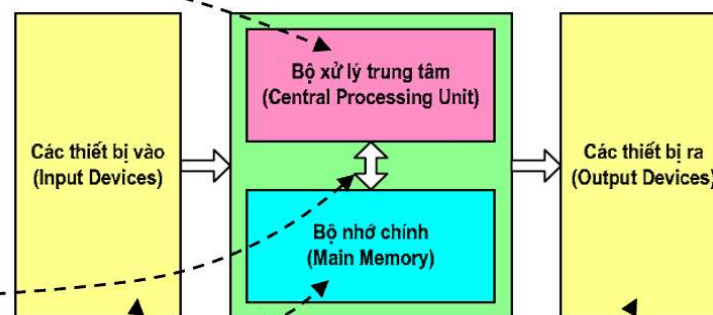
- Memory Hierarchy Model
- Memory Classification and Memory Circuit Organization
- Main Memory: ROM & RAM
- Cache Memory



❖ Characteristics of computer memory

Vị trí

- Bên trong CPU:
 - Tập thanh ghi
- Bộ nhớ trong:
 - Bộ nhớ cache
 - Bộ nhớ chính
- Bộ nhớ ngoài: các thiết bị nhớ



Dung lượng

- Độ dài từ nhớ - tính bằng bit
- Số lượng từ nhớ



❖ Characteristics of computer memory

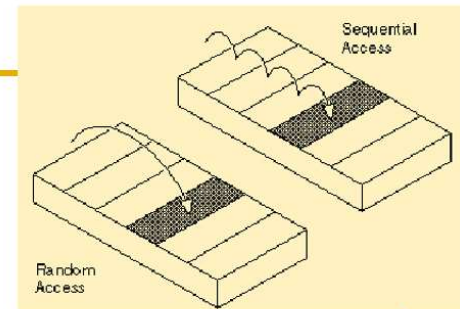
Đơn vị truyền

- Từ nhớ - Word
- Khối nhớ - Block



Phương pháp truy nhập

- Truy nhập tuần tự (băng từ)
- Truy nhập trực tiếp (các loại đĩa)
- Truy nhập ngẫu nhiên (bộ nhớ bán dẫn)
- Truy nhập liên kết (cache)



❖ Characteristics of computer memory

Đặc tính vật lý

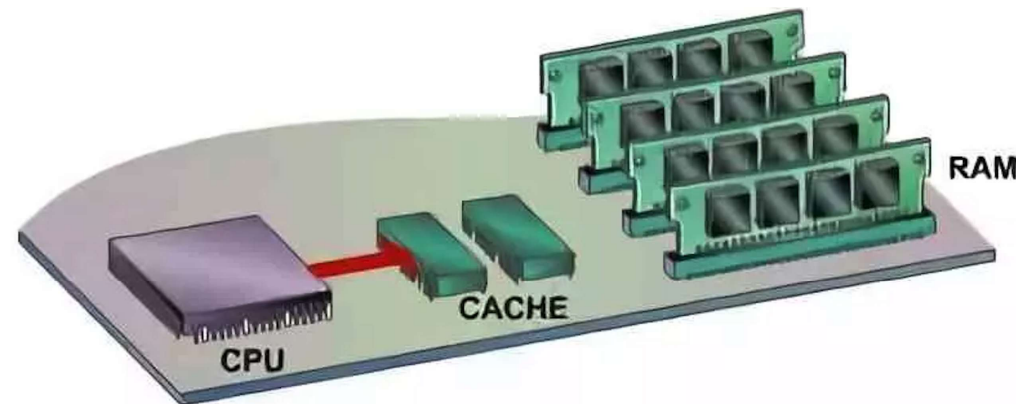
- Khả biến / Không khả biến (volatile / nonvolatile)
- Xoá được / Không xoá được

Kiểu vật lý

- Bộ nhớ bán dẫn
- Bộ nhớ từ
- Bộ nhớ quang

❖ Memory hierarchy components

- CPU Registers:
 - Very small size (a few tens of bytes to a few KB)
 - Very fast speed, access time around 0.25 ns, expensive
 - Temporarily stores input and output data for instructions
- Cache:
 - Small size (64KB to 16MB)
 - Fast speed, access time around 1-5 ns, expensive
 - Stores instructions and data for the CPU. Also known as "smart memory"



❖ Memory hierarchy components

■ Main Memory:

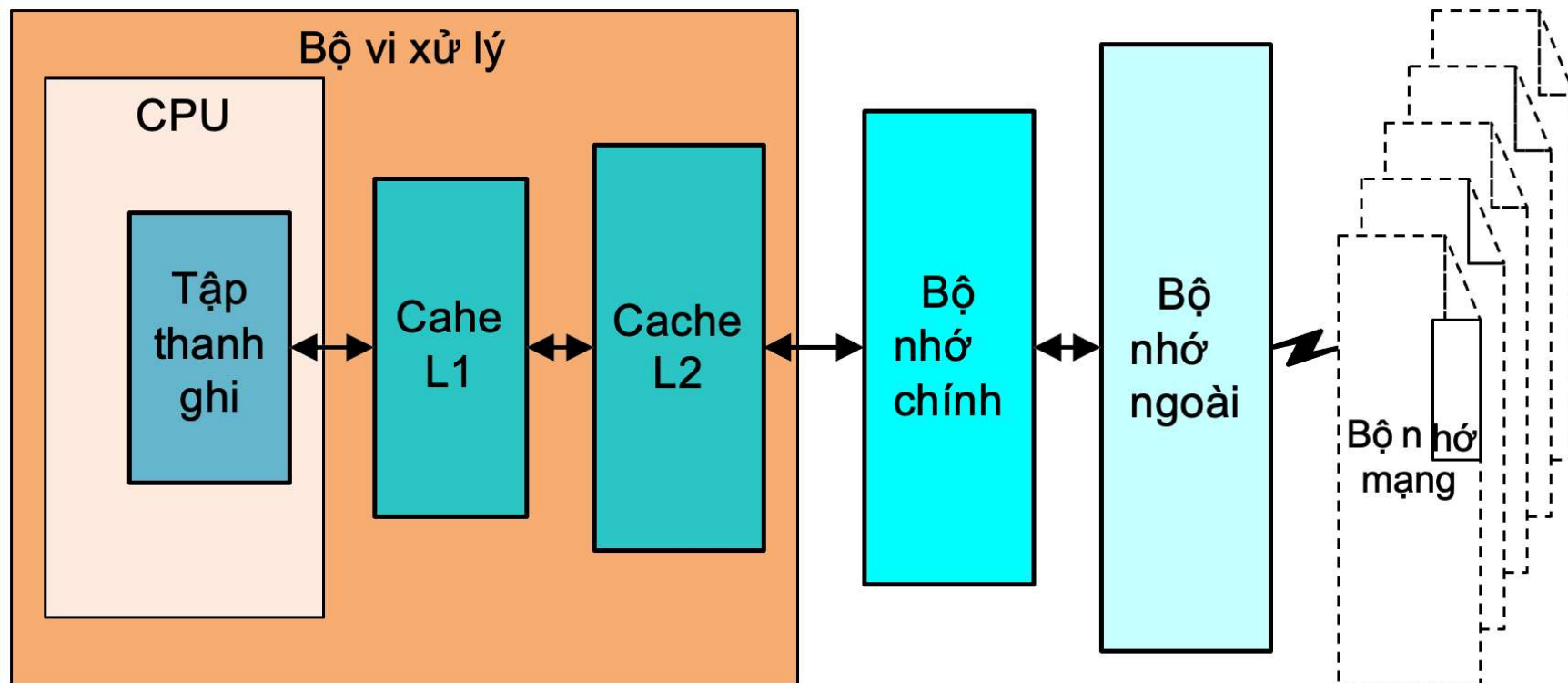
- Large size, capacity from 256MB to 4GB for 32-bit systems
- Slow speed, access time from 50-70 ns, Inexpensive
- Stores instructions and data for the system and user

■ Secondary Memory:

- Very large size, capacity from 20GB to 1000GB
- Very slow speed, access time around 5ms, very low cost
- Store large amounts of data
in files for extended periods

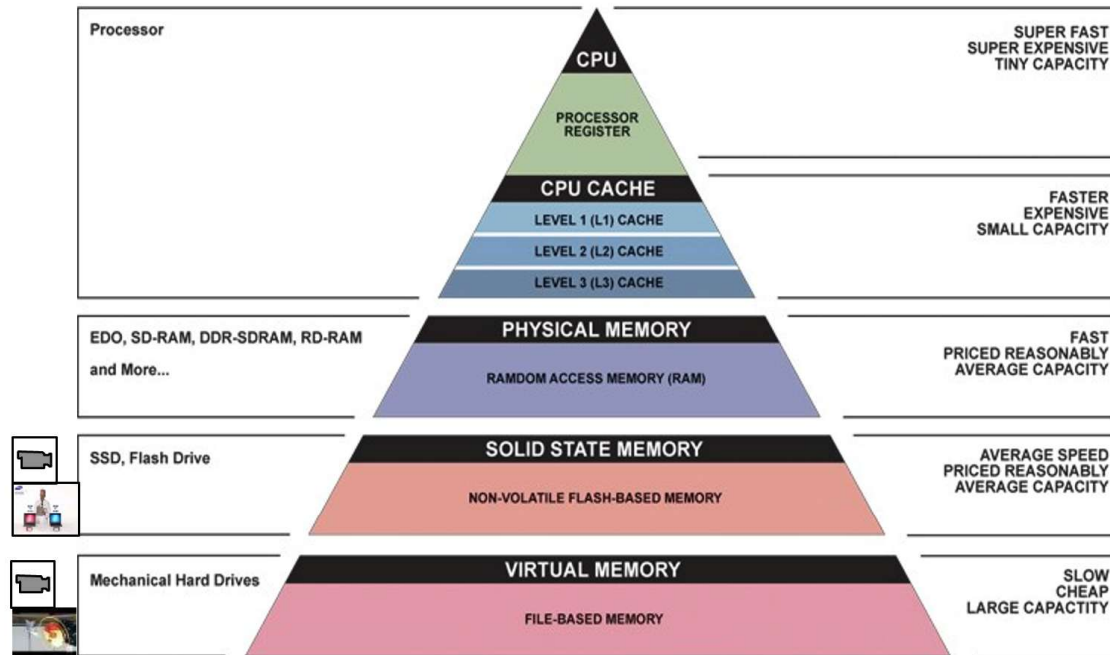


❖ Memory hierarchy components

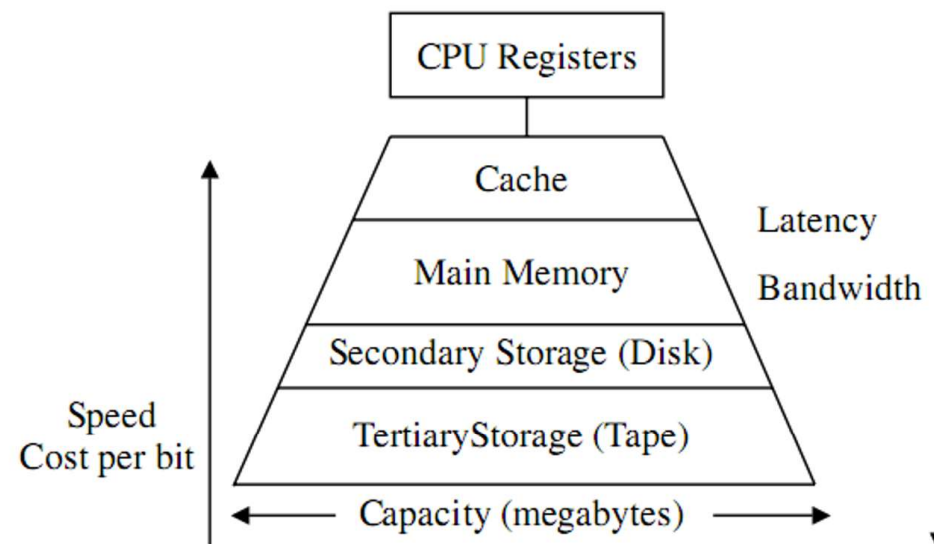


Memory hierarchy model

❖ Memory hierarchy components



▲ Simplified Computer Memory Hierarchy

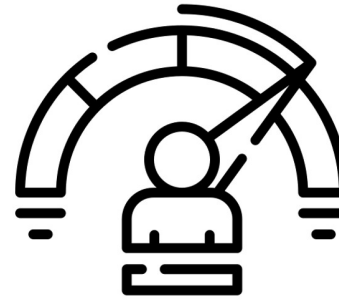


❖ Memory hierarchy components

	Access type	Capacity	Latency	Bandwidth	Cost/MB
CPU registers	Random	64–1024 bytes	1–10 ns	System clock rate	High
Cache memory	Random	8–512 KB	15–20 ns	10–20 MB/s	\$500
Main memory	Random	16–512 MB	30–50 ns	1–2 MB/s	\$20–50
Disk memory	Direct	1–20 GB	10–30 ms	1–2 MB/s	\$0.25
Tape memory	Sequential	1–20 TB	30–10,000 ms	1–2 MB/s	\$0.025

❖ Objective:

- Improved system performance:
 - Balancing high-speed CPUs with lower-speed main and secondary memory
 - Average CPU data access time from the memory system is nearly equal to cache access time
- Reduced production costs:
 - More expensive components are used in smaller capacities
 - Cheaper components are used in larger capacities
 - The total cost of a hierarchical memory system is lower than a non-hierarchical memory system of the same speed



❖ Based on access type:

- Random Access Memory (RAM)
- Serial Access Memory (SAM)
- Read-Only Memory (ROM)

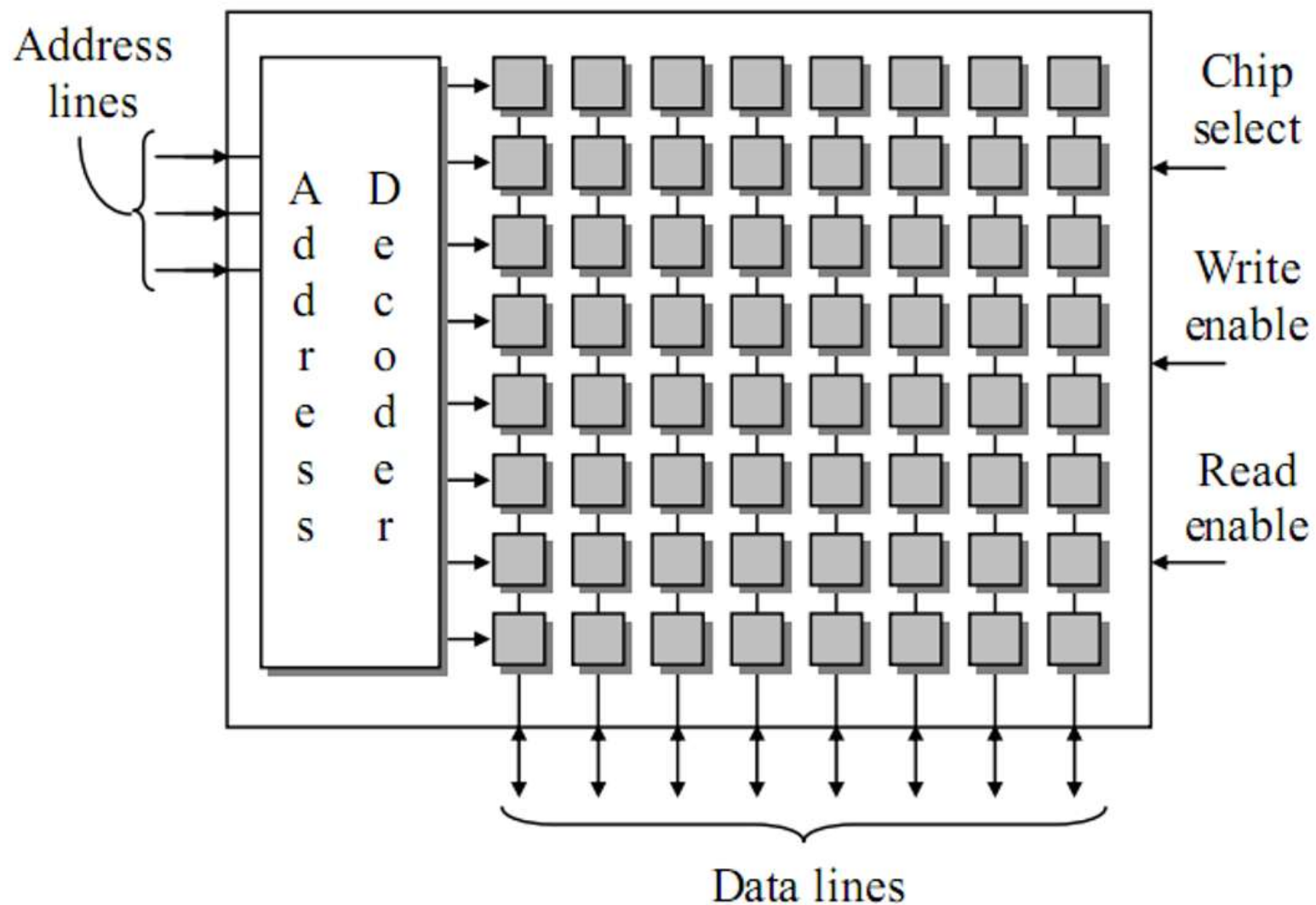
❖ Based on information retention/storage capacity:

- Volatile memory: stored information is lost when the power is turned off
- Stable memory: stored information is retained when the power is turned off

❖ Based on manufacturing technology:

- Semiconductor memory: ROM, RAM, SSD, USB, ...
- Magnetic memory: HDD, FDD, tape
- Optical memory: CD, DVD

Memory circuit organization



❖ Address lines:

- Address lines connected to bus A
- Transmit address signals from the CPU to the memory circuit

❖ Address decoder:

- Address decoder
- Uses the address to select and activate the memory location/line to be accessed

❖ Data lines:

- Data lines connected to bus D
- Transmit data from memory to the CPU and vice versa

❖ Chip select CS:

- Chip select signal pin
- The memory chip is activated when $CS=0$. Normally, the CPU only works with one memory chip at a time

❖ Write enable WE:

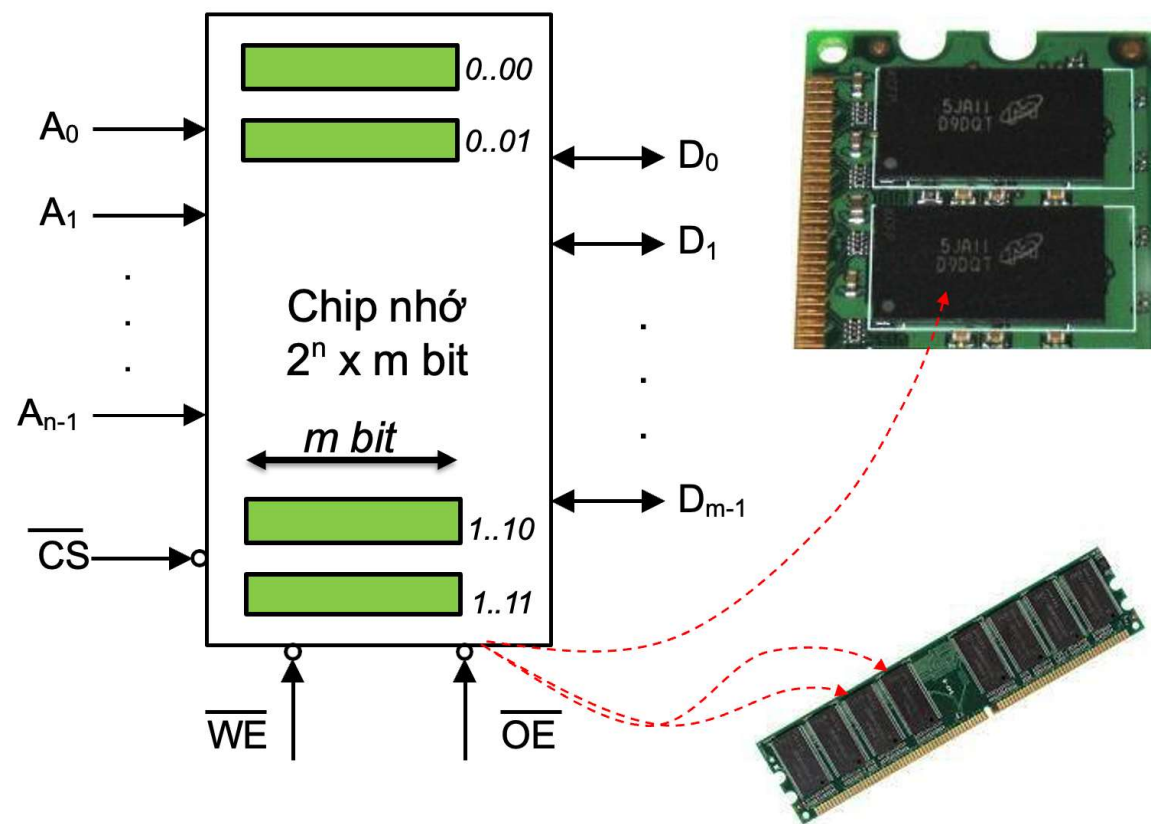
- Write enable signal pin
- Allows writing to the memory line when $WE = 0$

❖ Read enable RE:

- Read enable signal pin
- Allows reading data from the memory line when $RE = 0$

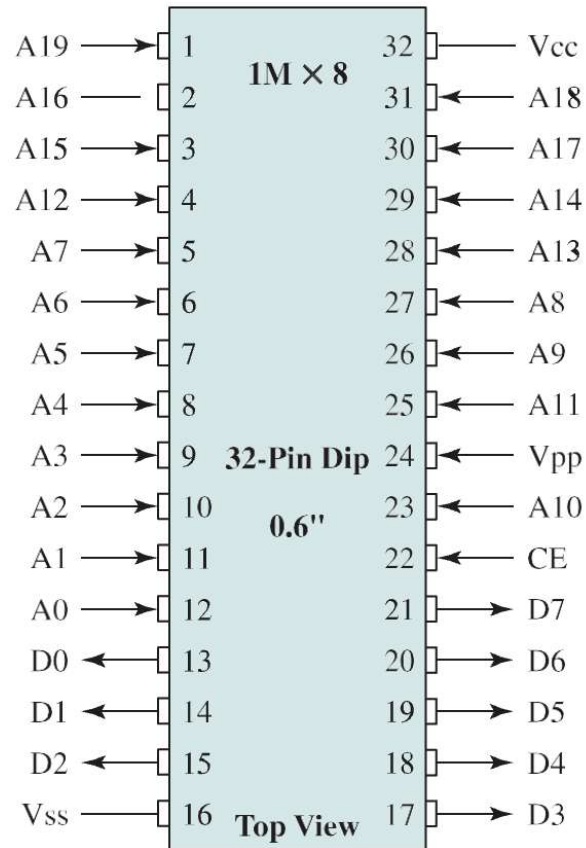
❖ Basic diagram of a memory chip:

- Address lines: $A_{n-1} - A_0$
 $\rightarrow 2^n$ memory words
- Data lines: $D_{m-1} - D_0$
 $\rightarrow$ memory word length = m bits
- Memory chip capacity
 $\rightarrow 2^n \times m$ bits

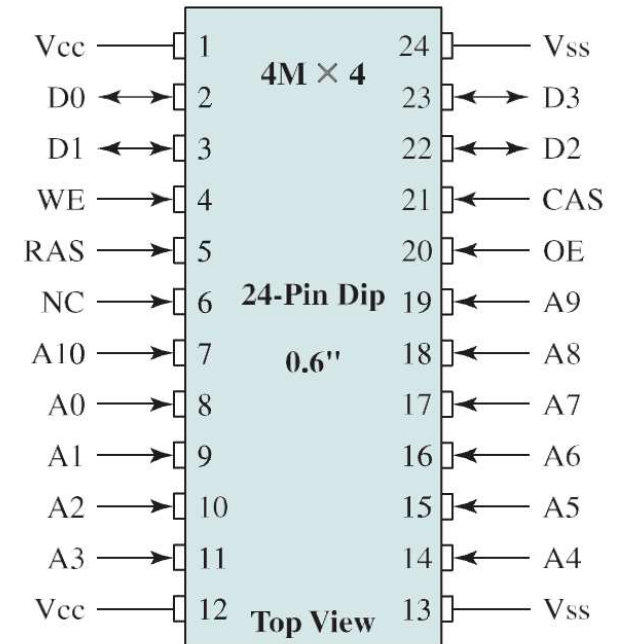


Memory circuit organization

❖ Examples:



(a) 8-Mbit EPROM



(b) 16-Mbit DRAM

- RAS → Row address latch
- OE → Enable data output to bus
- Vss → Ground (0V)

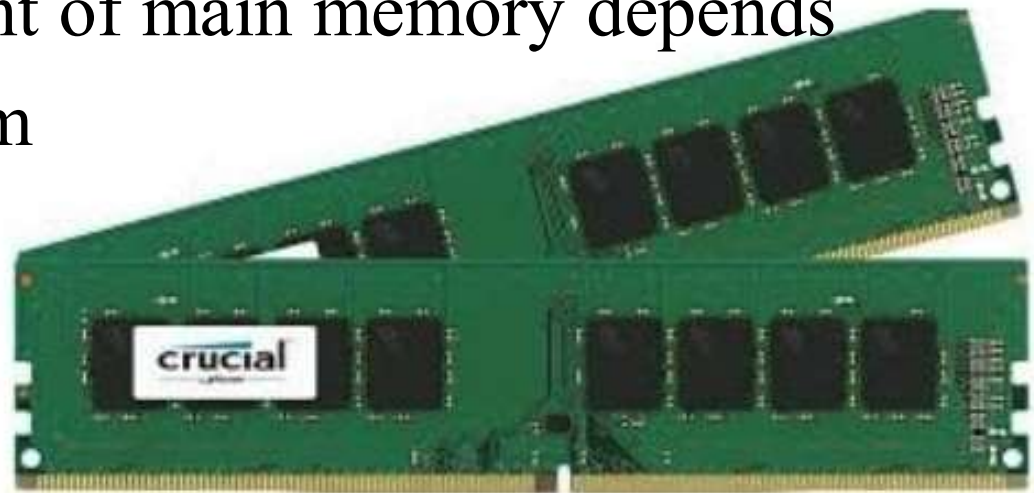
CAS → Column address latch
Vcc → Positive power supply

❖ Semiconductor memory:

Kiểu bộ nhớ	Tiêu chuẩn	Khả năng xóa	Cơ chế ghi	Tính khả biến
Read Only Memory (ROM)	Bộ nhớ chỉ đọc	Không xóa được	Mặt nạ	Không khả biến
Programmable ROM (PROM)			Bằng điện	
Erasable PROM (EPROM)	Bằng tia cực tím, cả chip			
Electrically Erasable PROM (EEPROM)	Bằng điện, mức từng byte			
Flash memory	Bằng điện, từng khối			
Random Access Memory (RAM)	Bộ nhớ đọc - ghi	Bằng điện, mức từng byte	Bằng điện	Khả biến

❖ Basic characteristics of internal memory:

- Contains running programs and data currently in use
- Exists on every computer system
- Consists of memory locations directly addressed by the CPU
- The capacity of main memory is smaller than the memory address space managed by the CPU.
- The logical management of main memory depends on the operating system



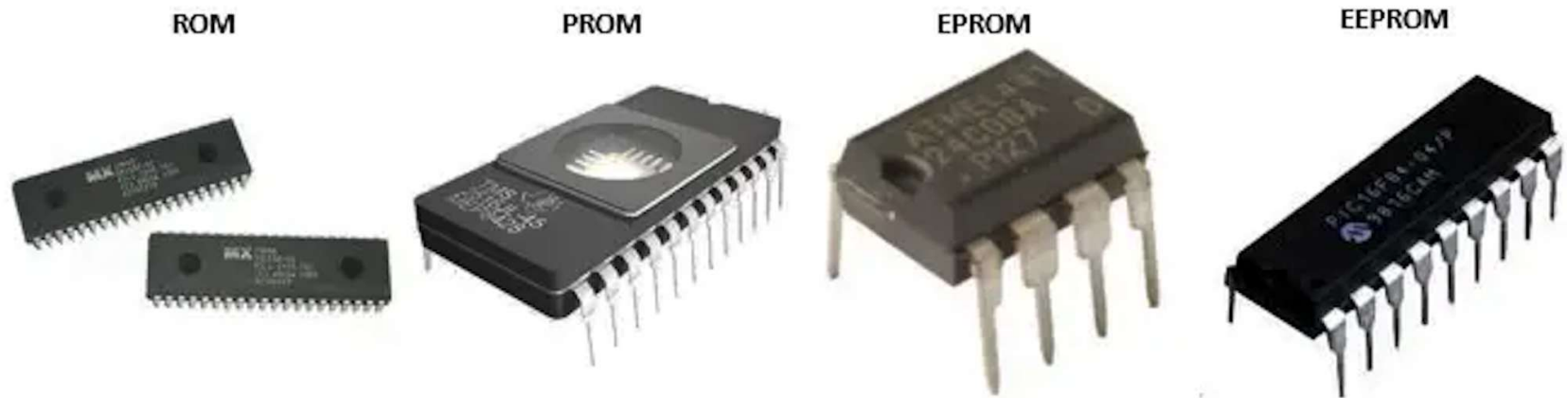
❖ ROM

- ROM (Read-Only Memory): is read-only memory.
- Information is written to ROM using special devices or methods.
- ROM is stable memory: all information is retained even when the power supply is lost.
- It is semiconductor memory: each memory cell is a semiconductor gate.
- It is commonly used to store system information: hardware information and BIOS.



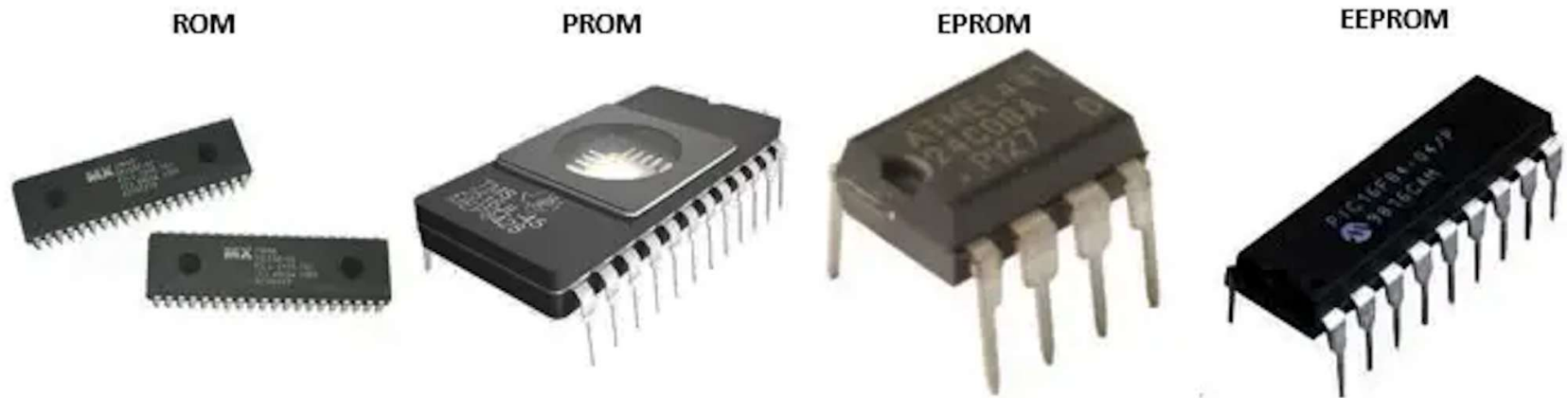
❖ Types of ROM

- Ordinary ROM:
 - First generation ROMs
 - Use ultraviolet light to write information
- PROM (Programmable ROM):
 - Programmable ROM
 - Information can be written to PROM using a special device called a PROM programmer



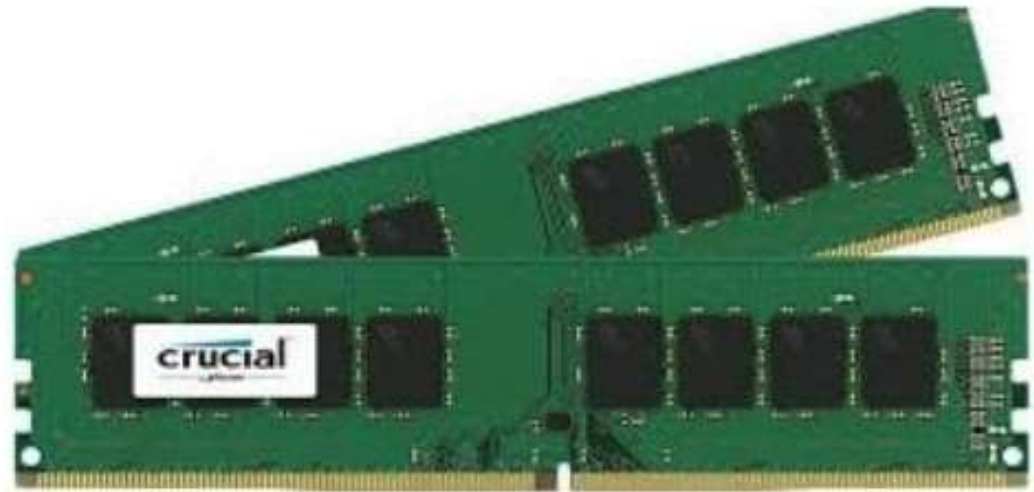
❖ Types of ROM

- EPROM (Erasable programmable read-only memory):
 - Programmable and erasable ROM
 - Information in EPROM can be erased by exposing it to high-intensity ultraviolet light
- EEPROM (Electrically Erasable PROM):
 - PROM whose information can be erased electrically
 - Information can be written using specialized software



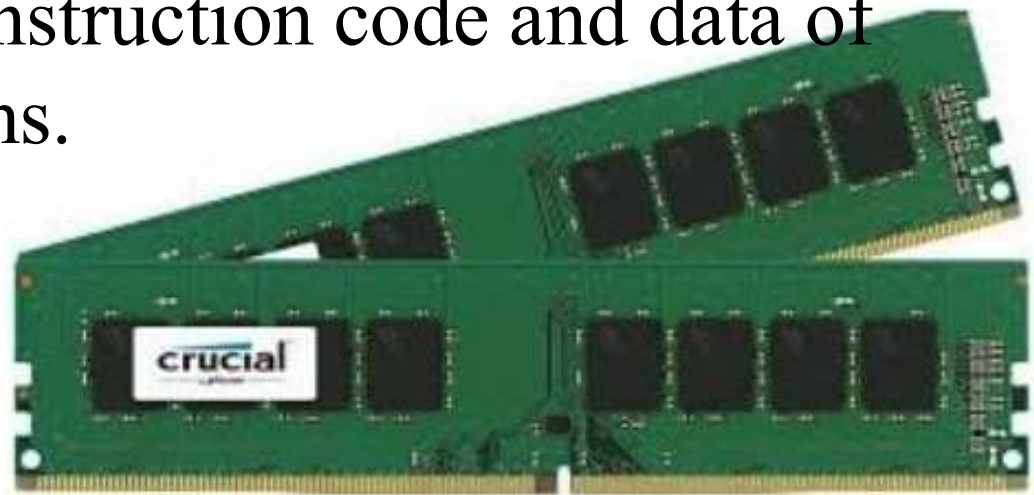
❖ RAM

- RAM (Random Access Memory) is random access memory.
 - Memory cells can be accessed randomly and not in any particular order.
 - The access speed of memory cells is comparable.
- It is volatile memory: all stored information is lost when the power is turned off.



❖ RAM

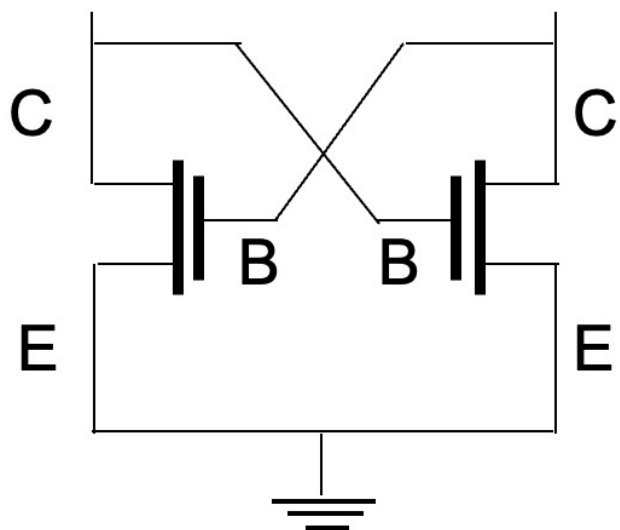
- It is semiconductor memory. Each memory cell is a semiconductor gate.
- It is used to store system and user information.
 - System information: hardware and operating system information.
 - User information: instruction code and data of application programs.



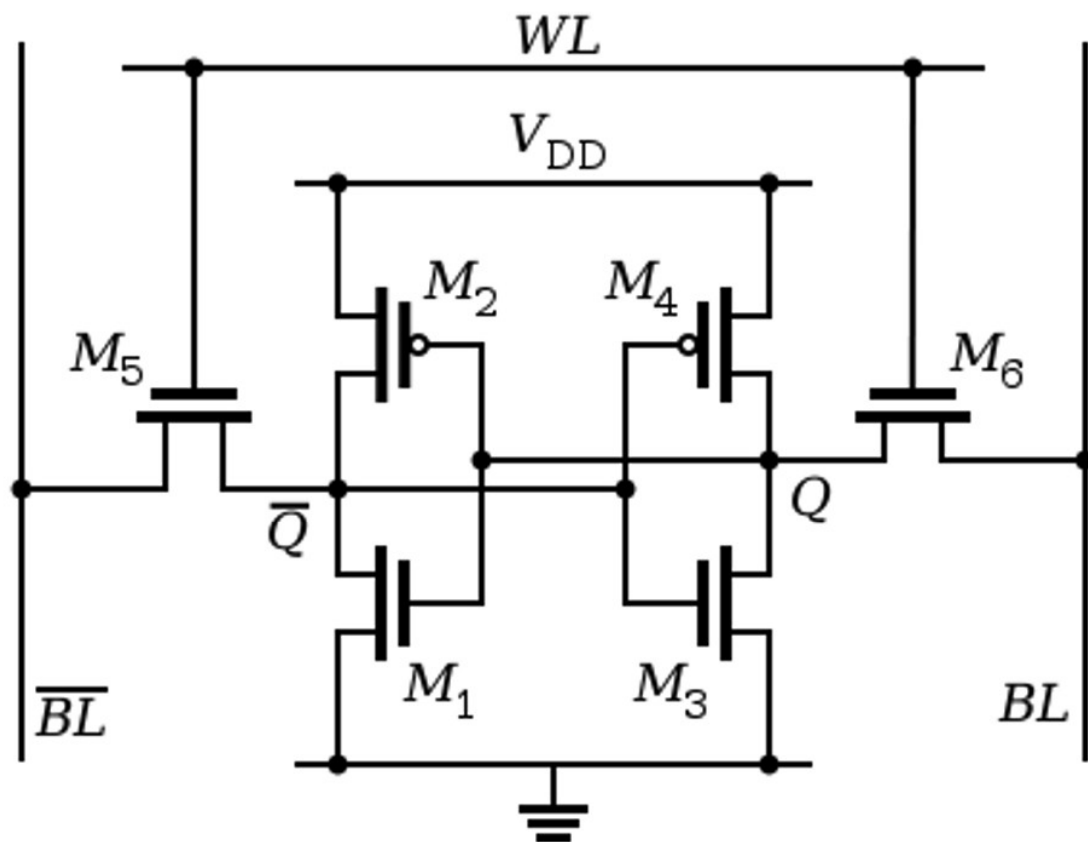
❖ Classification of RAM memory:

- Static RAM (SRAM):
 - Each bit of SRAM is a flip-flop circuit
 - Information stored in SRAM bits is always stable and does not require periodic refresh
 - SRAM is faster but more expensive than DRAM
- Dynamic RAM (DRAM):
 - Each bit of DRAM is based on a capacitor
 - Information stored in DRAM bits is not stable and must be refreshed periodically
 - DRAM is slower than SRAM but cheaper

❖ Characteristics of SRAM:



Một mạch lật đơn giản

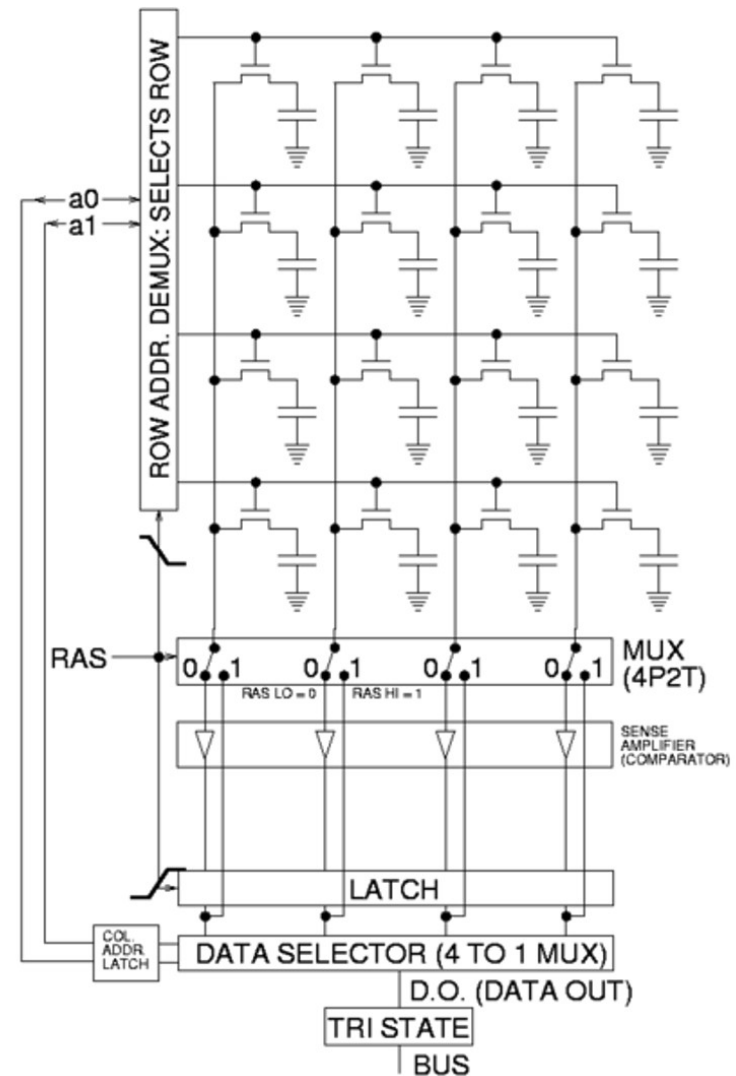
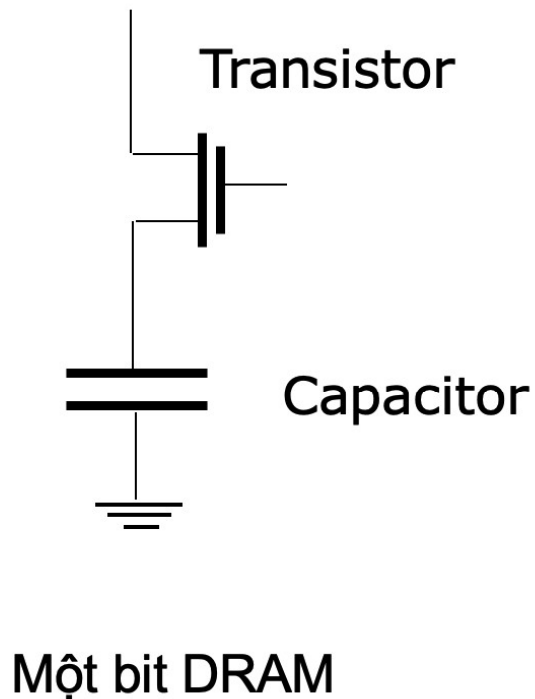


Một ô nhớ SRAM loại 6T

❖ Characteristics of SRAM:

- SRAM uses a bistable latching circuit to store 1 bit of information.
 - Each latch circuit storing 1 bit usually uses 6, 8, or 10 transistors (called 6T, 8T, or 10T circuits).
- SRAM usually has fast access speeds because:
 - The bits have a symmetrical structure.
 - SRAM memory circuits accept all address pins at the same time (no multiplexing).
- SRAM is expensive because:
 - It uses more transistors per bit than DRAM.
 - Due to its more complex internal structure, the density of SRAM is lower than DRAM.

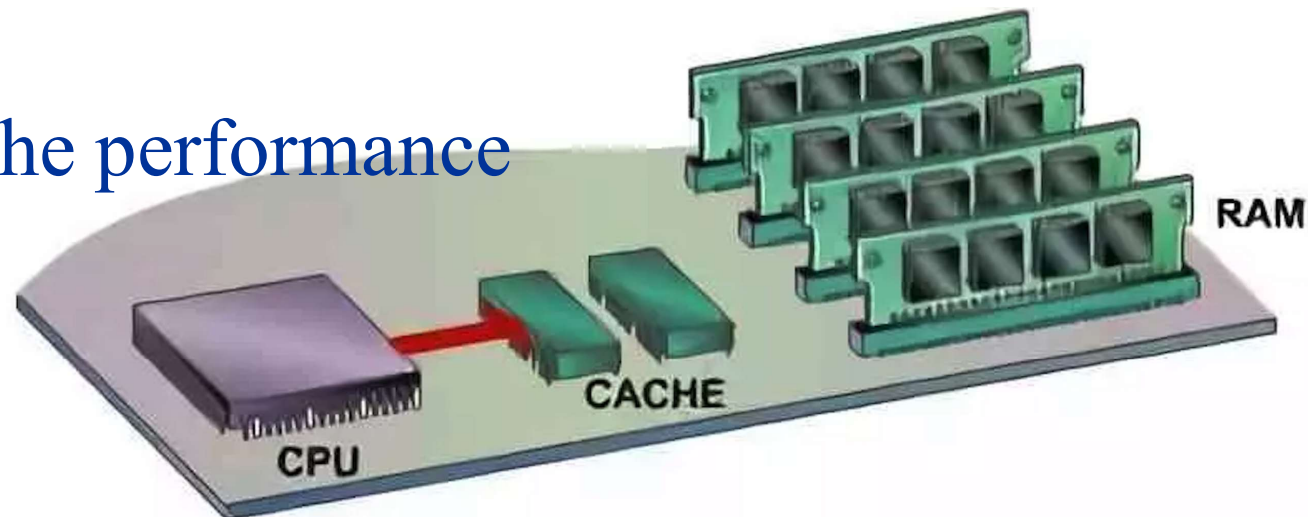
❖ Characteristics of DRAM:



❖ Characteristics of DRAM:

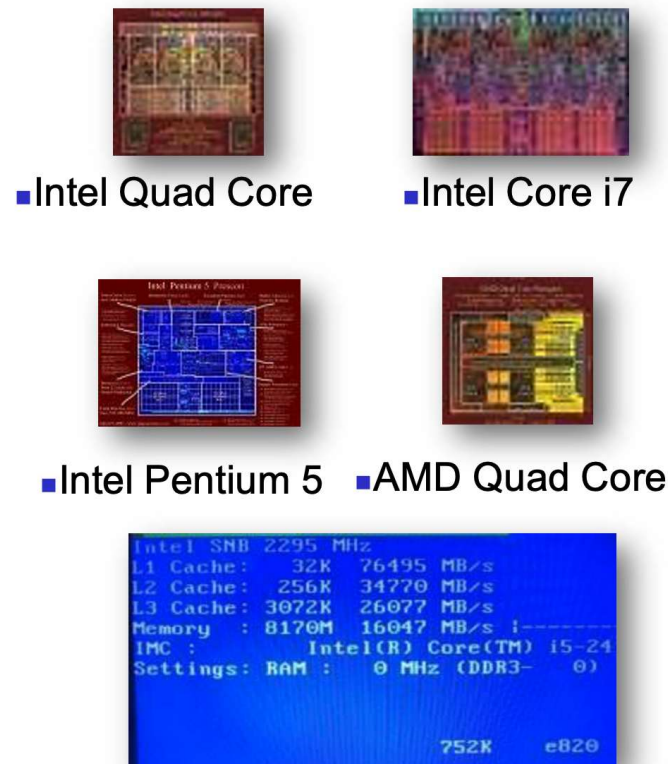
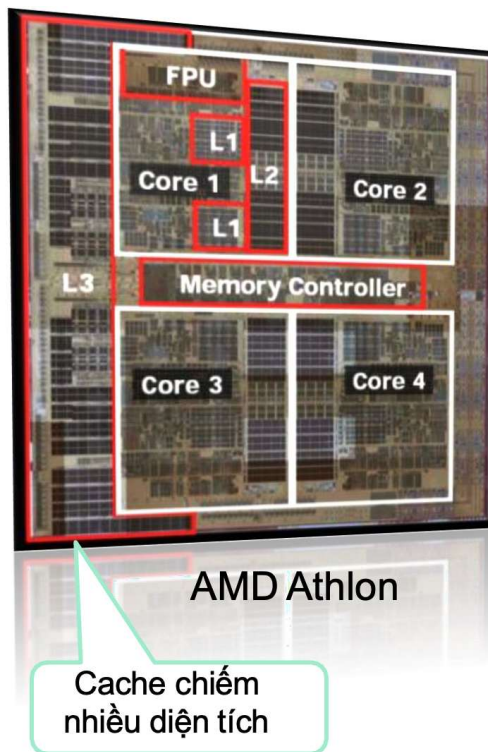
- DRAM bits are based on a capacitor and a transistor.
 - The two charge levels of the capacitor represent two logic levels:
- No charge: level 0
- Fully charged: level 1
- Because capacitors often self-discharge, the charge in the capacitor tends to dissipate.
 - Information in the capacitor needs to be reloaded frequently to avoid data loss.
 - Reloading the capacitor is a refresh process and must be done periodically.

- What is a cache?
 - Role of caching
 - Basic operating principles of caching
- Cache architecture
 - Cache organization
 - Read/write operations in the cache
 - Alternative policies
- Ways to improve cache performance



❖ What is Cache Memory?

- Cache is a memory component in a computer's memory hierarchy.
 - It acts as an intermediary, transferring data from main memory to the CPU and vice versa.



- ❖ Cache location:
 - In older systems, the cache is usually located outside the CPU.
 - In newer CPUs, the cache is usually integrated into the CPU.
- ❖ Cache capacity is usually small:
 - In older systems: 16K, 32K, ..., 128K
 - In newer systems: 256K, 512K, 1MB, 2MB, ...
- ❖ Cache access speed is faster than main memory speed.
- ❖ Cache cost (per bit) is usually more expensive than main memory.

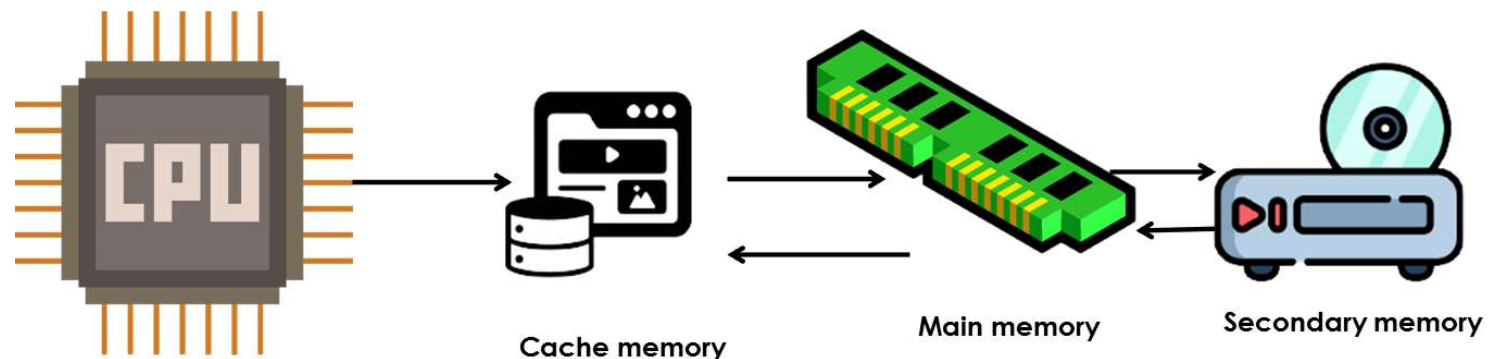


❖ Examples of cache operations:

1. The CPU requests the contents of a memory location.
2. The CPU checks the cache for this data.
3. If it exists, the CPU retrieves the data from the cache (quickly).
4. If it doesn't exist, it reads the memory block containing the data from main memory into the cache.
5. Then, it transfers the data from the cache to the CPU.

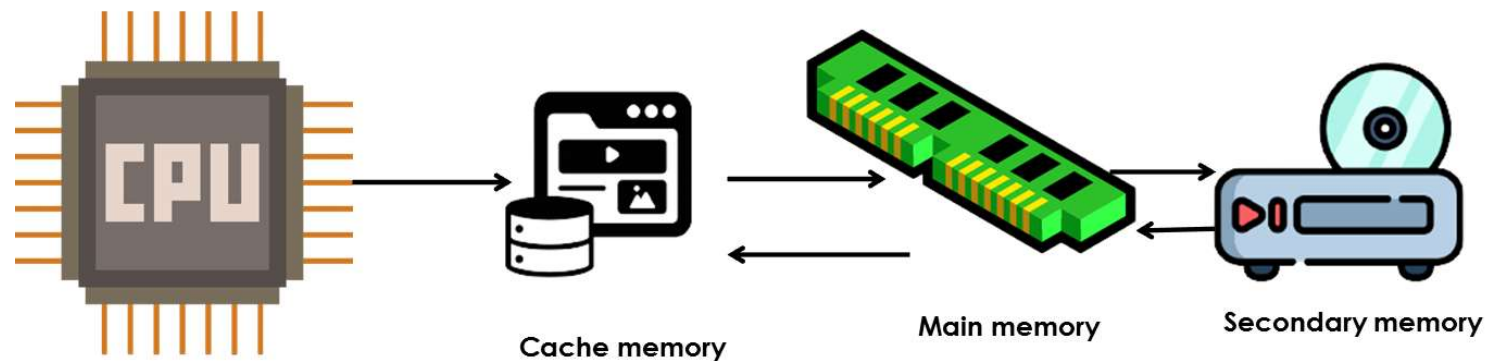
❖ The Role of Cache Memory

- Improves System Performance:
 - Balances high-speed CPUs with low-speed main memory (reduces the number of direct CPU accesses to main memory)
 - Average CPU access time to system memory is nearly equal to cache access time



❖ The Role of Cache Memory

- Reduces Production Costs:
 - If two systems have the same performance, the system with cache will be cheaper
 - If two systems have the same cost, the system with cache will be faster



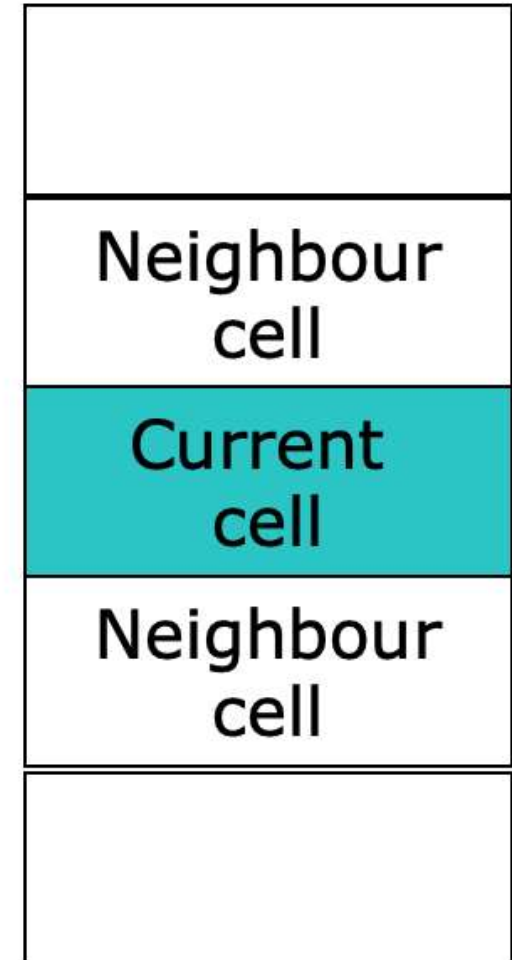
❖ Operating Principles of Cache Memory

- Cache is considered intelligent memory:
 - Cache can anticipate CPU requests for instructions and data
 - Necessary data and instructions are transferred in advance from main memory to the cache → CPU only accesses the cache → reduces memory access time
- Cache operates based on two fundamental principles:
 - Spatial locality principle
 - Temporal locality principle



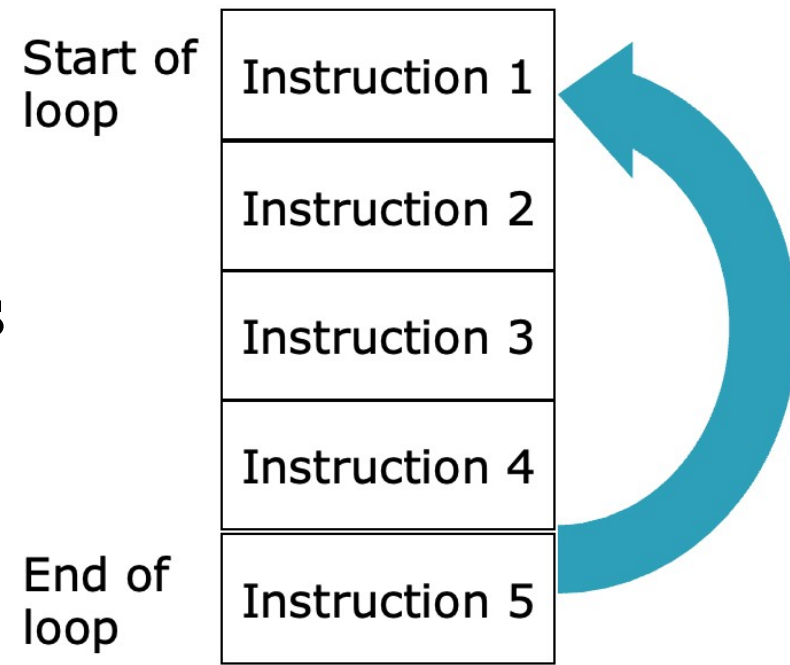
❖ Local (neighborhood) in terms of space:

- If a memory location is accessed, the probability that nearby locations will be accessed in the near future is high.
- Applies to data and instructions that are sequentially ordered in the program.
- Instructions in a program are usually sequential, so the cache reads a block of instructions in memory, which includes elements surrounding the location of the currently accessed element.



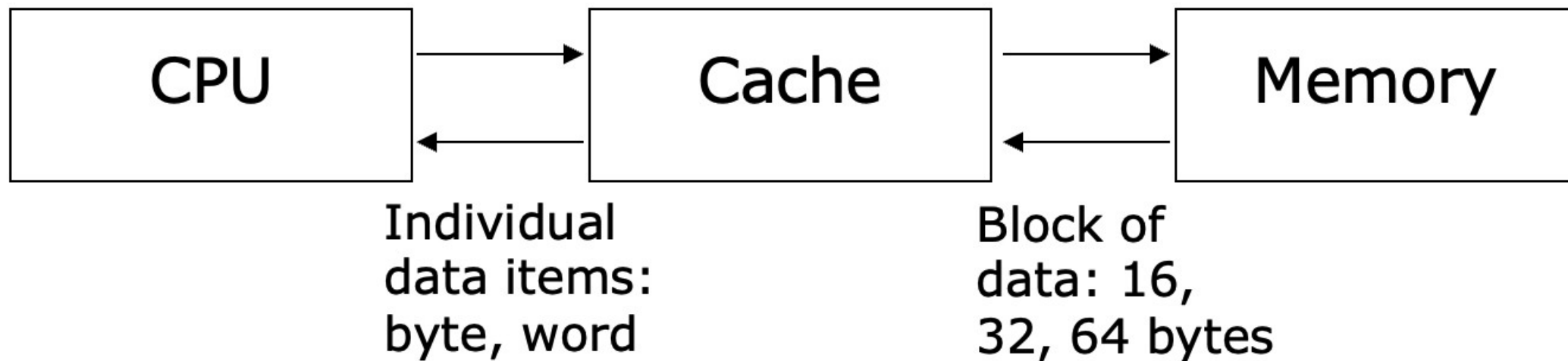
❖ Local (neighborhood) in time:

- If a memory location is accessed, the probability that it will be accessed again in the near future is high.
- Applies to data items and instructions within a loop.
- The cache reads a block of data in memory containing all the elements within the loop.



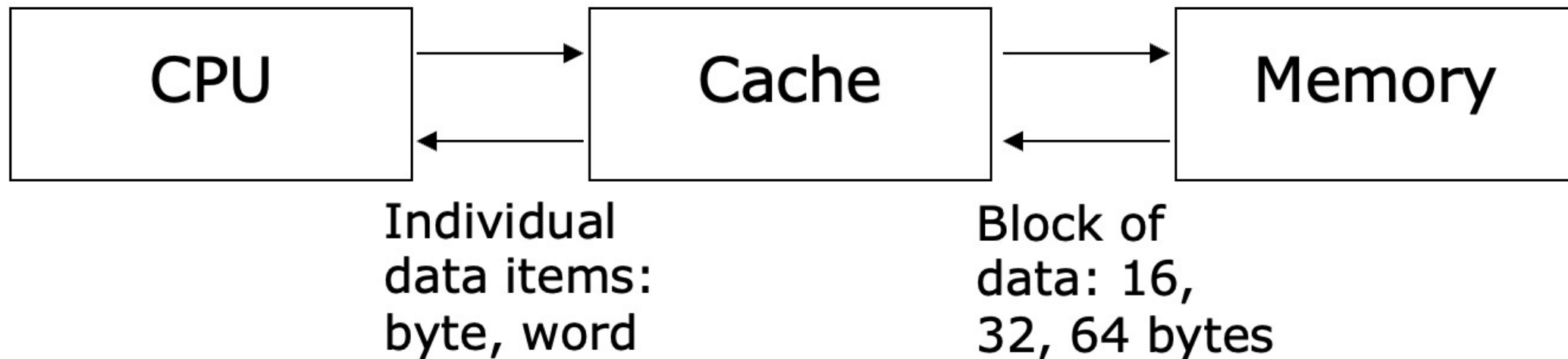
❖ Data Exchange:

- CPU reads/writes individual data items to/from the cache.
- Cache reads/writes blocks of data to/from memory.



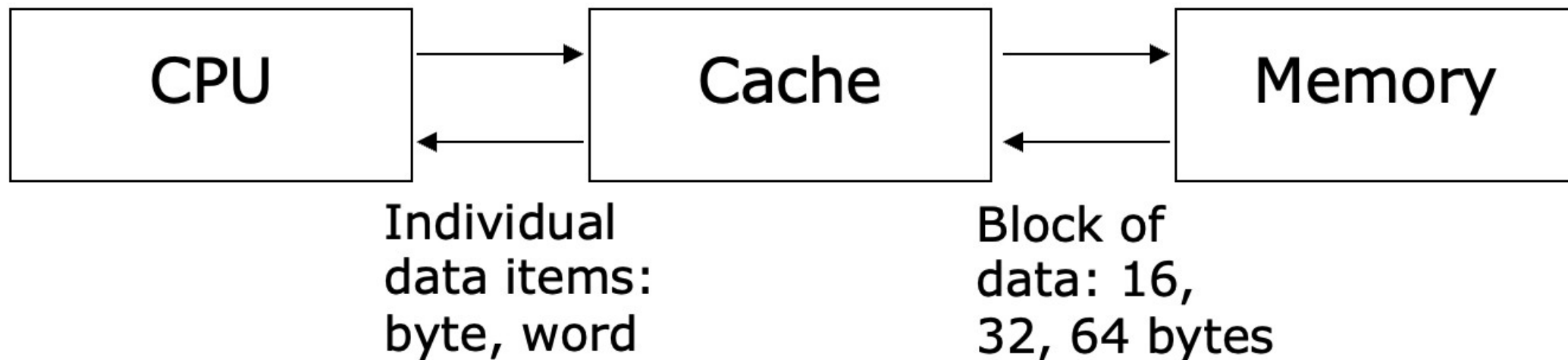
❖ Data Exchange:

- A hit is the event where the CPU accesses data/instructions found in the cache.
 - The probability of a hit occurring is called the hit coefficient H : $0 \leq H \leq 1$.
 - The higher H , the better the cache.



❖ Data Exchange:

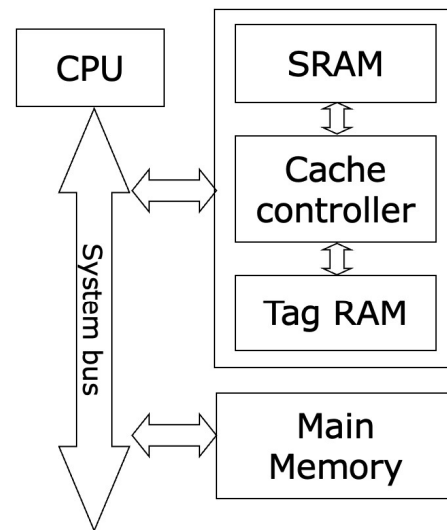
- A miss is the event where the CPU accesses data/instructions not found in the cache.
 - The likelihood of a miss occurring is called the miss coefficient or $1-H$: $0 \leq (1 - H) \leq 1$.
 - A low miss coefficient indicates high cache efficiency.



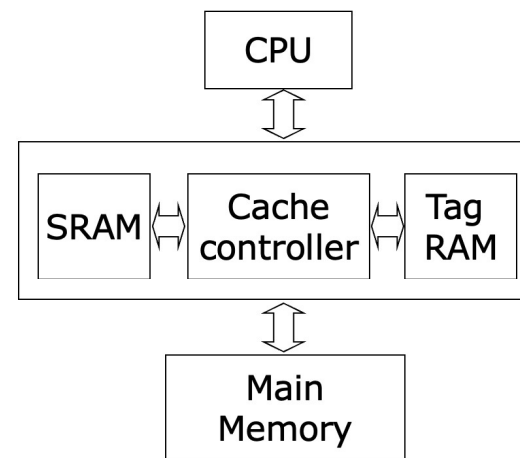
❖ Cache Architecture and Organization:

Cache architecture refers to where the cache is located in relation to the CPU and memory. There are two cache layout architectures:

- Look-aside architecture: Peer-to-peer architecture
- Look-through architecture: Hierarchical architecture



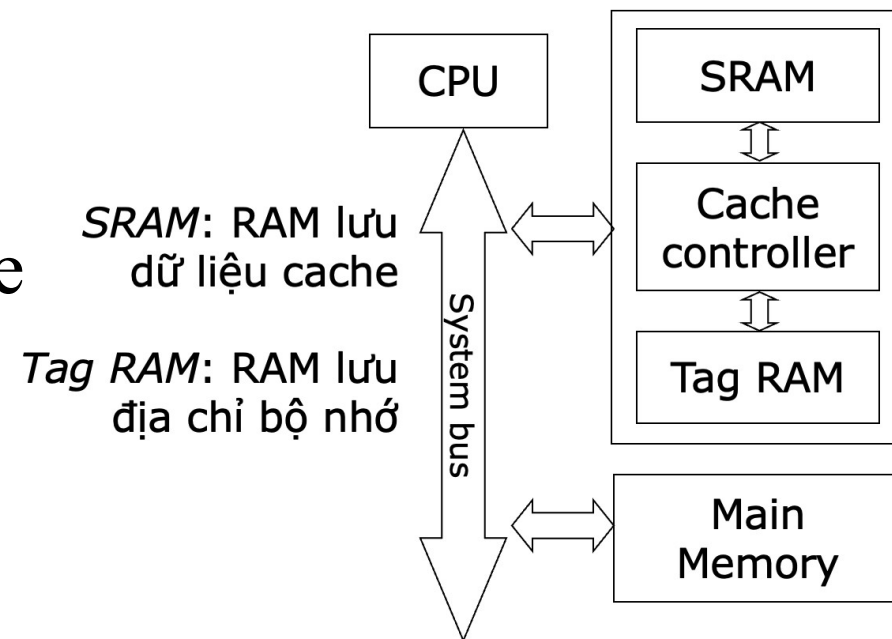
Look aside



Look through

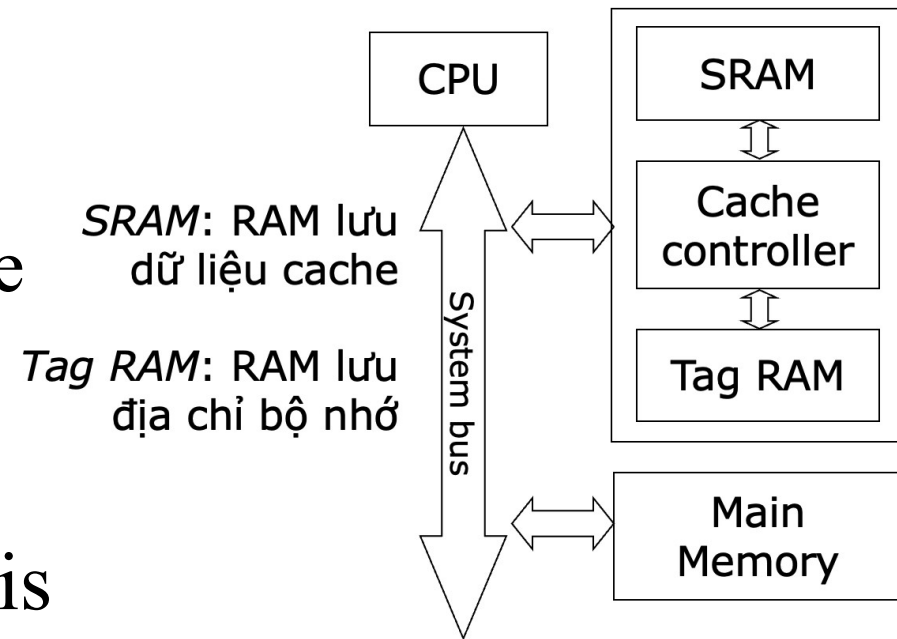
❖ Look-Aside Architecture:

- Cache and memory are both connected to the system bus.
- Cache and main memory "see" the CPU bus cycle at the same time.
- Advantages:
 - Simple design
 - Fast misses: when the CPU doesn't find a data item in the cache, it simultaneously searches in main memory for the same clock cycle.



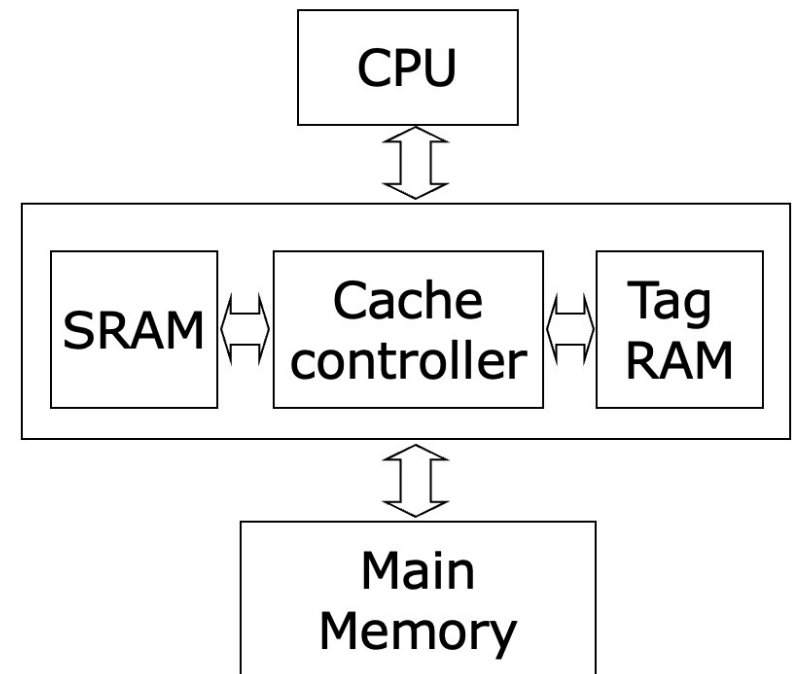
❖ Look-Aside Architecture:

- Cache and memory are both connected to the system bus.
- Cache and main memory "see" the CPU bus cycle at the same time.
- Disadvantages:
 - Slow hits: because the cache is connected to the CPU using the system bus, which has a low operating frequency and narrow bandwidth.



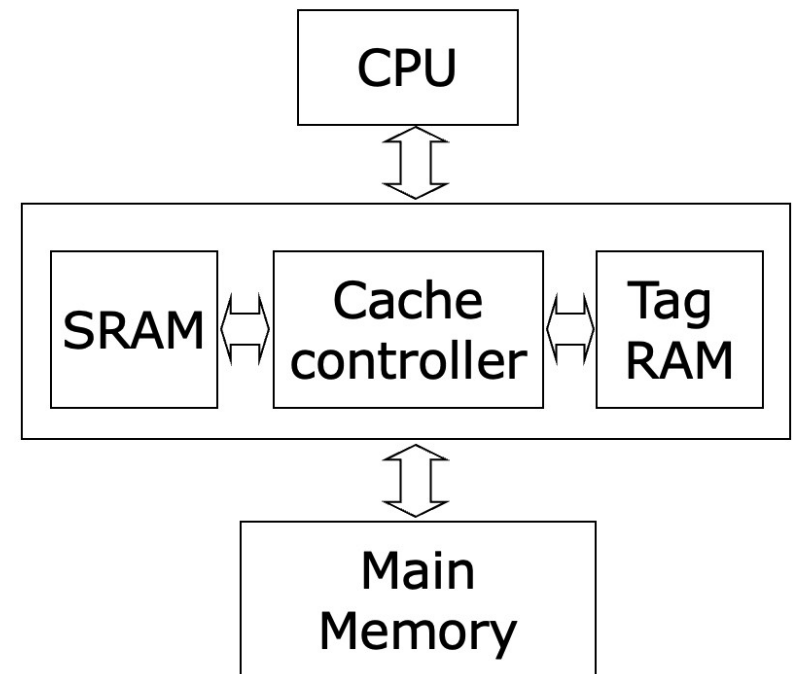
❖ Look-through architecture:

- Cache is located between the CPU and main memory
- The cache “sees” the CPU bus cycle first, then it “transmits” it to main memory
- Advantages:
 - Fast hits: The cache connects to the CPU via a dedicated, high-speed bus with large bandwidth



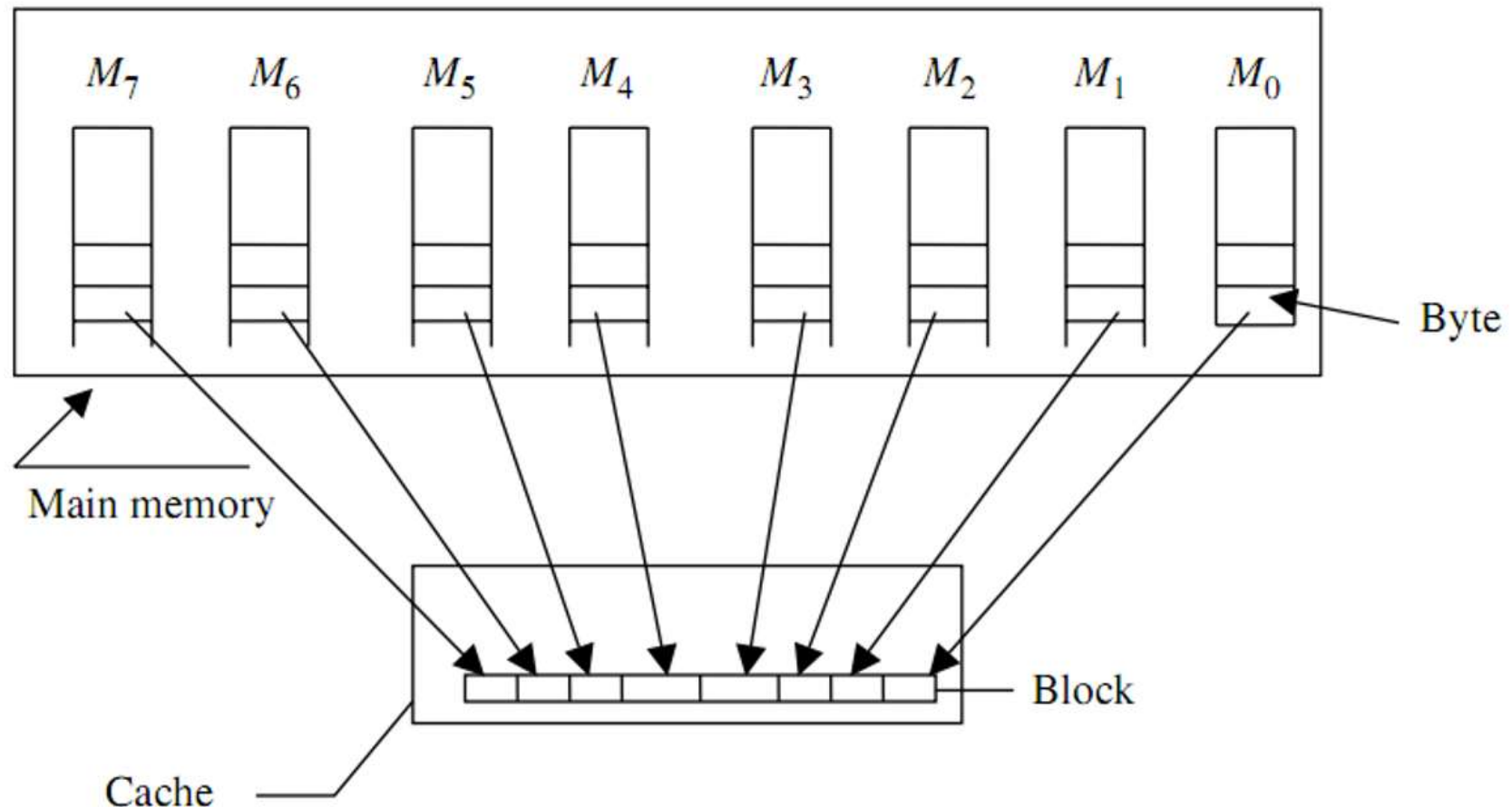
❖ Look-through architecture:

- Cache is located between the CPU and main memory
- The cache “sees” the CPU bus cycle first, then it “transmits” it to main memory
- Disadvantages:
 - Expensive
 - Complex design
 - Slow misses: When the CPU cannot find the data in the cache, it searches for it in memory at the next clock cycle



Cache Organization

❖ Cache Organization: Cache organization addresses how cache and main memory work together.

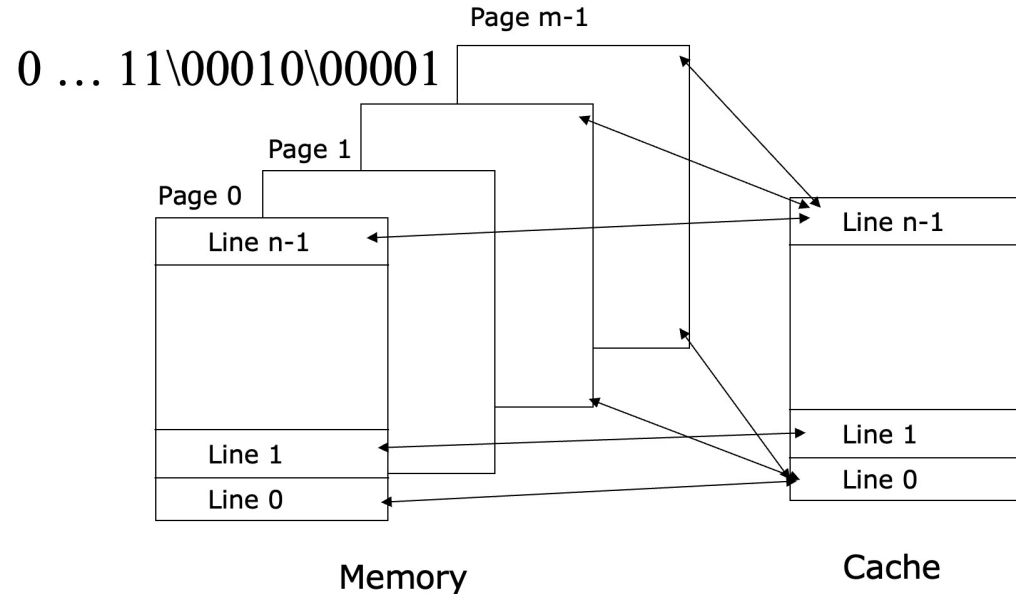


❖ Cache Organization Techniques:

- Direct mapping:
 - Simple, fast
 - Fixed mapping
- Fully associative mapping:
 - Complex, slow
 - Flexible mapping
- Set associative mapping:
 - Complex
 - Fast, flexible mapping

❖ Direct mapping:

- Cache: Divided into n blocks or lines from Line 0 to Line $n-1$
- Memory:
 - Divided into m pages, from page 0 to page $m-1$
 - Each memory page has the same size as the cache
 - Each page has n lines, from Line 0 to Line $n-1$
- Mapping:
 - Line i of (page 0 to page $m-1$) is mapped to Line i of the cache



❖ The address field of a direct mapping:

- Tag (bit): is the address of the page in memory
- Line (bit): is the address of the line in a cache page
- Word (bit): is the address of the word in a line

Tag	Line	Word
-----	------	------

How to determine:

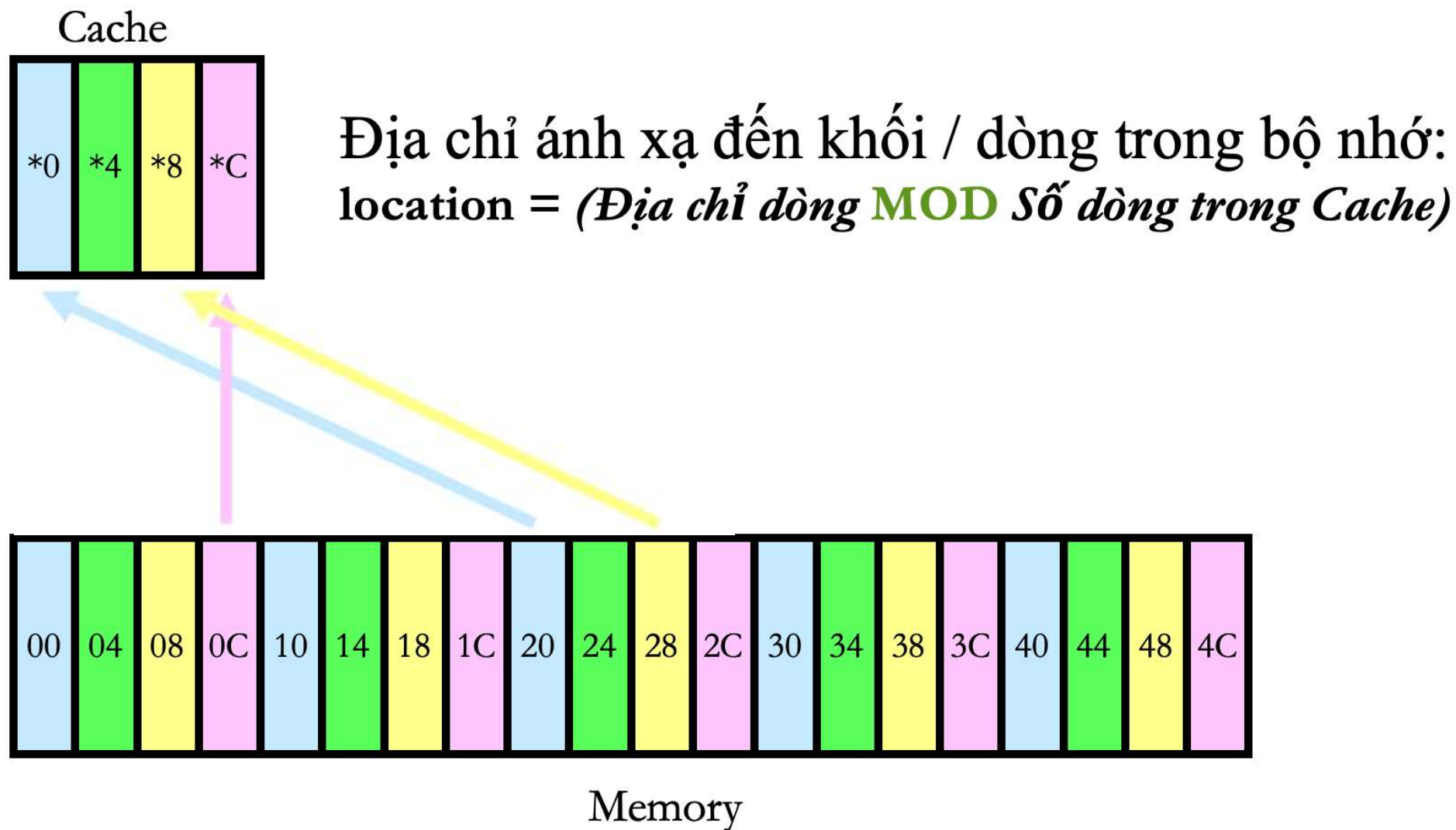
- $Tag + Line + Word = Address_Bus_Size(bit) = \log_2 \left(\frac{M_{Memory}}{[Cell]} \right)$
- Number of memory pages: $P = \frac{M_{Memory}}{[M_{Cache}]} \Rightarrow tag = \log_2 (P)$
- Number of lines in a page: $L = \frac{M_{Cache}}{[Line]} \Rightarrow line = \log_2 (L)$
- Number of memory cells in a line: $W = \frac{Line}{[Cell]} \Rightarrow word = \log_2 (W)$

❖ Exercise: Find the size of the address field using direct mapping, given:

- Memory size: 4GB
- Cache size: 1KB
- Line size: 32 bytes

Solution:

❖ Direct mapping cache access:



❖ Direct mapping cache access:

- Cache has Line = 4 and Word = 1 (4 blocks (1-word)).
- Find the number of Misses for this Cache memory using the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache access 1 - Mapping: block 0 $\rightarrow 0 \bmod 4 = 0$

Line 0 trống: write Mem[0]

Mem Block	Cache Hit/Miss
0	Miss

	Cache
Line 0	Mem[0]
Line 1	
Line 2	
Line 3	

❖ Direct mapping cache access:

- Cache has Line = 4 and Word = 1 (4 blocks (1-word)).
- Find the number of Misses for this Cache memory using the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache access 2 - Mapping: block 8 $\rightarrow 8 \bmod 4 = 0$

Mem Block	Cache Hit/Miss
0	Miss
8	

Line 0
Line 1
Line 2
Line 3

Cache
Mem[0]

Line 0 chứa Mem[0]. Ghi đè Mem[8] lên Line 0

Mem Block	Cache Hit/Miss
0	Miss
8	Miss

Line 0
Line 1
Line 2
Line 3

Cache
Mem[8]

❖ Direct mapping cache access:

- Cache has Line = 4 and Word = 1 (4 blocks (1-word)).
- Find the number of Misses for this Cache memory using the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache access 3 - Mapping: block 0 $\rightarrow 0 \bmod 4 = 0$

8	MISS
0	Miss

Line 1

Line 2

Line 3

Line 0 chứa Mem[8]. Ghi đè Mem[0] lên Line 0

Mem Block	Cache Hit/Miss
0	Miss
8	Miss
0	

Line 0

Line 1

Line 2

Line 3

Cache
Mem[0]

❖ Direct mapping cache access:

- Cache has Line = 4 and Word = 1 (4 blocks (1-word)).
- Find the number of Misses for this Cache memory using the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache access 4 - Mapping: block 6 $\rightarrow 6 \bmod 4 = 2$

8	MISS
0	Miss
6	Miss

Line 1	
Line 2	Mem[6]
Line 3	

Line 2 trống: Ghi Mem[6] lên Line 2

Mem Block	Cache Hit/Miss
0	Miss
8	Miss
0	Miss
6	

	Cache
Line 0	Mem[0]
Line 1	
Line 2	
Line 3	

❖ Direct mapping cache access:

- Cache has Line = 4 and Word = 1 (4 blocks (1-word)).
- Find the number of Misses for this Cache memory using the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache access 5 - Mapping: block 8 $\rightarrow 8 \bmod 4 = 2$

8	MISS
0	Miss
6	Miss
8	Miss

Line 1	
Line 2	Mem[6]
Line 3	

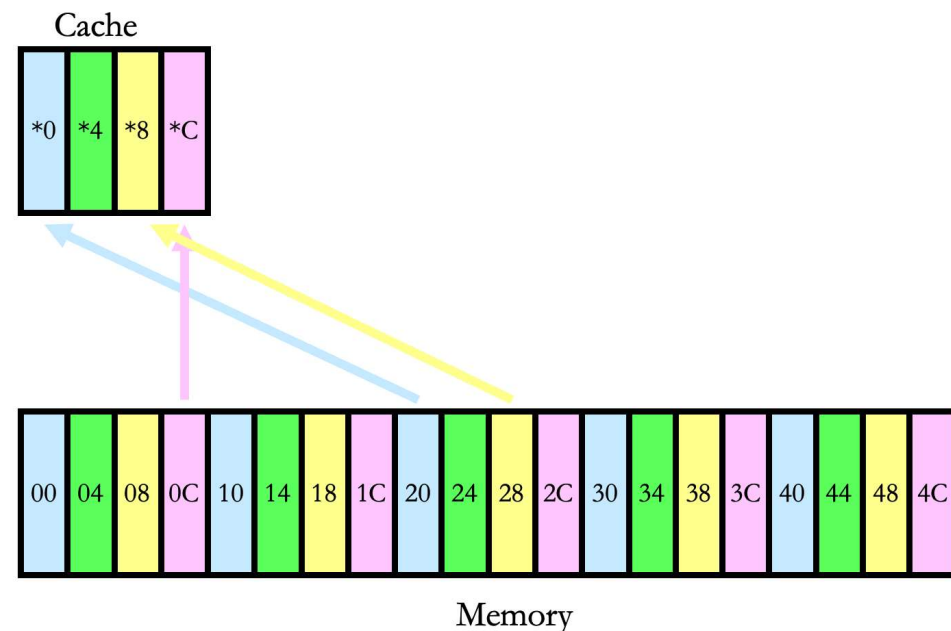
Line 0 chứa Mem[0]: Ghi đè Mem[8] lên Line 0

Mem Block	Cache Hit/Miss
0	Miss
8	Miss
0	Miss
6	Miss
8	

Cache	
Line 0	Mem[8]
Line 1	
Line 2	Mem[6]
Line 3	

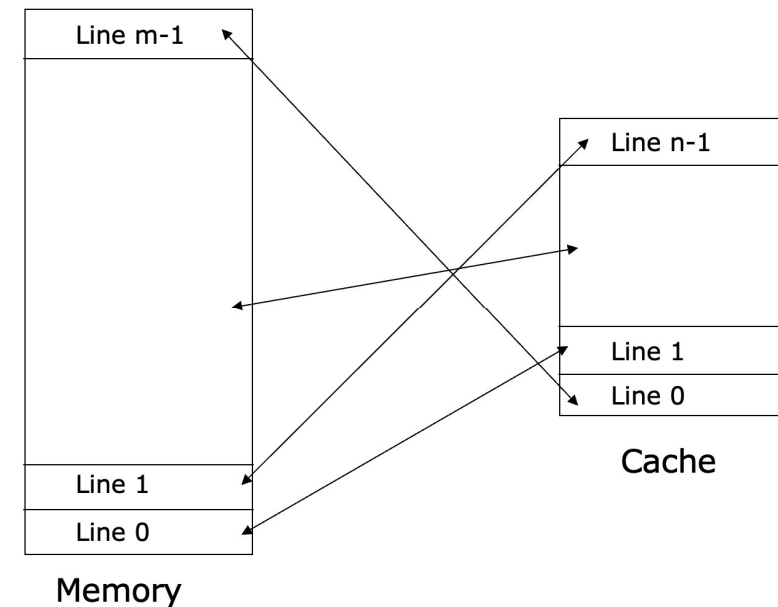
❖ Comments on direct-mapped cache

- Advantages:
 - Simple design
 - Fast because of fixed mapping: knowing the memory address allows for very quick retrieval the cache
- Disadvantages Points:
 - Because the mapping is fixed, the likelihood of conflicts is high.
 - Low hit rate.



❖ Fully associative mapping

- Cache: Divided into n blocks or lines from Line0 to Linen-1
- Memory:
 - Divided into m lines, from Line0 to Linem-1
 - The size of each cache line is equal to the size of a memory line
 - The number of lines in memory can be much larger than the number of cache lines ($m \gg n$)
- Mapping: Line i of memory is mapped to any Line j of cache



❖ The address field of Fully associative mapping

- Tag (bit): is the address of the line in memory
- Word (bit): is the address of the word in memory within a line



Where

- $\text{Tag} + \text{Word} = \text{Address_Bus_Size (bit)} = \log_2 \left(\frac{M_{Memory}}{[Cell]} \right)$
- Number of memory lines: $L = \frac{M_{Memory}}{[Line]} \Rightarrow tag = \log_2 (L)$
- Number of memory cells in a line:

$$W = \frac{[Line]}{[Cell]} \Rightarrow Word = \log_2 (W)$$

- ❖ Exercise: Find the size of the address field using Fully associative mapping, knowing:
- Memory size: 4GB
 - Cache size: 1KB
 - Size of a line: 32 bytes

❖ Fully associative mapping

- The cache has Word = 1.
- Find the number of misses of this cache with the CPU's block/line memory access sequence: 0, 8, 0, 6, 8.

Quy tắc thay thế dòng cache: Thay thế dòng ít được sử dụng gần đây nhất LRU

Mem Block	Cache Hit/Miss

Cache

❖ Fully associative mapping

- The cache has Word = 1.
- Find the number of misses of this cache with the CPU's block/line memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 1:

Line 0 trống: Ghi Mem[0] vào Line 0

Mem Block	Cache Hit/Miss
0	Miss

Cache
Mem [0]

❖ Fully associative mapping

- The cache has Word = 1.
- Find the number of misses of this cache with the CPU's block/line memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 2:

Dòng Line 1-3 trống: Ghi Mem[8] vào Line 1

Mem Block	Cache Hit/Miss
0	Miss
8	Miss

Cache
Mem [0]
Mem[8]

❖ Fully associative mapping

- The cache has Word = 1.
- Find the number of misses of this cache with the CPU's block/line memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 3:

Dòng Line 0 chứa Mem[0]

Mem Block	Cache Hit/Miss
0	Miss
8	Miss
0	Hit

Cache

Mem[0]
Mem[8]

❖ Fully associative mapping

- The cache has Word = 1.
- Find the number of misses of this cache with the CPU's block/line memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 4:

Ghi Mem[6] vào Line 2

Mem Block	Cache Hit/Miss
0	Miss
8	Miss
0	Hit
6	Miss

Cache	
Mem[0]	
Mem[8]	
Mem[6]	

❖ Fully associative mapping

- The cache has Word = 1.
- Find the number of misses of this cache with the CPU's block/line memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 5:

Mem Block	Cache Hit/Miss
0	Miss
8	Miss
0	Hit
6	Miss
8	Hit

Line 1 chứa Mem[8]

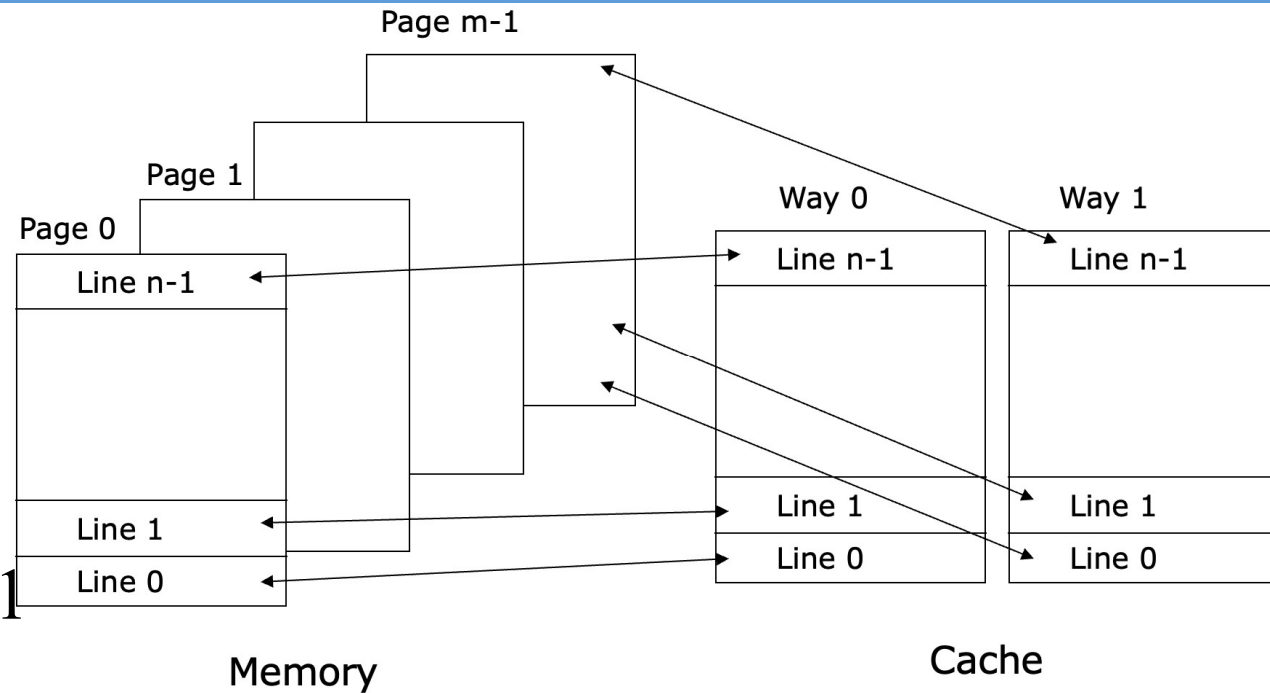
Cache
Mem[0]
Mem[8]
Mem[6]

❖ Fully associative mapping

- Advantages:
 - Fewer conflicts due to flexible mapping
 - Higher hit rate
- Disadvantages:
 - Slow because it has to search for memory addresses in the cache
 - Complex because it has n additional address comparators in the cache
- Often used for small cache sizes

❖ Set associative mapping

- Divided into k ways of equal size
- Each way is divided into n blocks or lines, from Line 0 to Line $n-1$
- Memory:
 - Divided into m pages, from page 0 to page $m-1$
 - The size of one page is equal to the size of one way of the cache
 - Each page is divided into n blocks or lines, from Line 0 to Line $n-1$



❖ Set associative mapping

- Mapping: Pages are mapped to Ways (full mapping):
- A memory page can be mapped to any way in the cache.
 - Page lines are mapped to way lines (direct - fixed mapping):
- Line i of page i is mapped to Line i of way j ;

Tag	Set	Word
-----	-----	------

❖ The address field of Set associative mapping

- Tag (bit): is the address of the page in memory
- Set (bit): is the address of the line in a memory page
- Word (bit): is the address of the word in a memory line

Tag	Set	Word
-----	-----	------

where

- $\text{Tag} + \text{Set} + \text{Word} = \text{Address_Bus_Size (bit)} = \log_2 \left(\frac{M_{\text{Memory}}}{[\text{Cell}]} \right)$
- Number of memory pages: $P = \frac{M_{\text{Memory}} \times \text{Way}}{M_{\text{Cache}}} \Rightarrow \text{tag} = \log_2 (P)$
- Number of lines in a page: $L = \frac{M_{\text{Cache}}}{[\text{Word}] \times \text{Way}} \Rightarrow \text{Set} = \log_2 (L)$
- Number of memory cells in a line: $W = \frac{[\text{Line}]}{[\text{Cell}]} \Rightarrow \text{Word} = \log_2 (W)$

- ❖ Exercise: Find the size of the address field using associative set mapping:
- Memory size: 4GB
 - Cache size: 1KB, 2 ways
 - Size of a line: 32 bytes

Solution:

❖ Set associative mapping

- Cache has Word = 1, Set = 2, 2-way.
- Find the number of misses in this cache with the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

Quy tắc thay thế dòng cache: thay thế way ít sử dụng nhất trong cùng một dòng

Mem Block	Cache Hit/Miss
0	

Set 0

Set 1

Way0	Way1

❖ Set associative mapping

- Cache has Word = 1, Set = 2, 2-way.
- Find the number of misses in this cache with the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 1:

Dòng Set 0 trống: Ghi Mem[0] vào Way0

Mem Block	Cache Hit/Miss
0	miss

Set 0

Set 1

Way0	Way1
Mem[0]	

❖ Set associative mapping

- Cache has Word = 1, Set = 2, 2-way.
- Find the number of misses in this cache with the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 2:

Mem Block	Cache Hit/Miss
0	miss
8	

Mem Block	Cache Hit/Miss
0	miss
8	miss

	Way0	Way1
Set 0	Mem[0]	
Set 1		

Dòng Set 0 trống: Ghi Mem[8] vào Way1

	Way0	Way1
Set 0	Mem[0]	Mem[8]
Set 1		

❖ Set associative mapping

- Cache has Word = 1, Set = 2, 2-way.
- Find the number of misses in this cache with the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 3:

8	miss
0	hit

Set 1		
-------	--	--

Dòng Set 0 Way 0 chứa Mem[0]

Mem Block	Cache Hit/Miss
0	miss
8	miss
0	

	Way0	Way1
Set 0	Mem[0]	Mem[8]
Set 1		

❖ Set associative mapping

- Cache has Word = 1, Set = 2, 2-way.
- Find the number of misses in this cache with the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

► Cache Access 4:

8	miss
0	hit
6	miss

Set 1		
-------	--	--

Ghi đề Mem[6] lên dòng Set 0 và Way1

Mem Block	Cache Hit/Miss
0	miss
8	miss
0	hit
6	

	Way0	Way1
Set 0	Mem[0]	Mem[6]
Set 1		

❖ Set associative mapping

- Cache has Word = 1, Set = 2, 2-way.
- Find the number of misses in this cache with the CPU's block/row memory access sequence: 0, 8, 0, 6, 8.

8	miss
0	hit
6	miss
8	miss

Mem Block	Cache Hit/Miss
0	miss
8	miss
0	hit
6	miss
8	

Set 1		
-------	--	--

Ghi đề Mem[8] vào dòng Set0 và Way0

	Way0	Way1
Set 0	Mem[8]	Mem[6]
Set 1		

❖ Read operations:

- Hit case: The requested data item is found in the cache
 - The data item is read from the cache into the CPU
 - Main memory is not involved
- Miss case:
 - The data item is read from memory into the cache
 - Then the data is transferred from the cache to the CPU
- Miss penalty: The time to access the data item is equal to the total time to access the cache and main memory

❖ Write operations:

- Hit case (message belongs to a block in the cache)
 - Write through: data is written to the cache and to memory simultaneously
 - Write back: data is first written to the cache, and the entire block containing it in the cache is written back to memory later when that block is replaced
- Miss case (message not in the cache)
 - Write allocate: data is first written to memory, and then the entire block containing it is read to the cache
 - Write non-allocate: data is only written to memory

- ❖ Why replace cache lines?
 - Mapping from one line in memory to another in the cache is usually a many-to-one mapping.
 - Multiple memory lines share a single cache line, so memory lines are loaded into the cache, used for a period of time, and then replaced by another line as the CPU requests information.
- ❖ Replacement policies define how cache lines are selected for replacement when new lines from memory need to be moved in.
- ❖ Cache line replacement strategies include three main strategies:
 - Random replacement
 - First In First Out (FIFO)
 - Least Recently Used (LRU) replacement

❖ Random Replacement

- Rows in the cache are randomly selected for replacement
- Simple
- High miss rate because this method does not consider which cache row is actually in use
 - If a cache row in use is replaced, a miss event occurs and the cache needs to be read again.

❖ First In First Out (FIFO)

- Based on the FIFO principle: the cache line read into the cache first will be selected for replacement first
- Lower miss rate compared to random methods
- Miss rate is still high because this method does not actually consider which block is currently in use
 - An “old” cache line may still be in use
- Complex implementation because it requires additional circuitry to monitor the order in which memory lines are loaded into the cache

❖ Least Recently Used (LRU) lines

- The least recently used cache lines will be selected for replacement
- Consideration of lines currently in use
- Lowest miss rate compared to random and FIFO methods
- Complex implementation due to the need for additional circuitry to monitor the usage frequency of cache lines

❖ Cache performance.

- Average access time of a cached memory system.

$$\begin{aligned}T_{\text{access}} &= (\text{hit cost}) + (\text{miss cost}) \\&= (\text{Hit cost}) + (\text{miss rate}) * (\text{miss penalty}) \\&= t_{\text{cache}} + (1 - H) * (t_{\text{memory}})\end{aligned}$$

Trong đó H là hệ số hit

Nếu $t_{\text{cache}} = 5\text{ns}$, $t_{\text{memory}} = 60\text{ns}$ và $H=80\%$, ta có:

$$t_{\text{access}} = 5 + (1 - 0.8) * (60) = 5 + 12 = 17\text{ns}$$

Nếu $t_{\text{cache}} = 5\text{ns}$, $t_{\text{memory}} = 60\text{ns}$ và $H=95\%$, ta có:

$$t_{\text{access}} = 5 + (1 - 0.95) * (60) = 5 + 3 = 8\text{ns}$$

❖ Factors affecting cache performance include:

- Cache size
- Multi-level caching
- Cache partitioning

Where cache size factor:

- Larger cache size:
 - Can store more data blocks in memory
 - Reduces the frequency of exchanging data blocks of different programs between memory and cache
 - Larger caches are slower than smaller caches:
 - Searching for memory locations in a larger space

❖ Factors affecting cache performance include:

- Cache size
- Multi-level caching
- Cache partitioning

Where cache size factor:

- Larger cache size:
 - Trend: caches are getting larger and larger
 - Better multi-tasking support
 - Better support for parallel processing
 - Better support for multi-core CPU systems

❖ Multi-level caching:

- Multi-level caching can better balance CPU and MEM speeds than single-level caching.
- In practice, caches usually have two levels: L1 and L2. Some caches have three levels: L1, L2, and L3.
- Reduces memory system costs.

	CPU	L1	L2	L3	Bộ nhớ chính
Cache 3 mức:	1ns	5ns	15ns	30ns	60ns
Cache 1 mức:	1ns	5ns			60ns

❖ Cache partitioning factors:

- Cache can be partitioned into instruction cache and data cache for better performance because:
- Data and instructions differ in locality:
 - Data is more time-local than instructions, while instructions are more spatially local.
- Instruction cache only supports read operations, while data cache supports both read and write operations, making cache optimization easier.
- Supports multiple read/write operations simultaneously, reducing resource conflicts.
- Combines other functions such as pre-decoding instructions into the instruction cache, resulting in better instruction processing.

❖ Cache partitioning factors:

- In practice, most L1 caches are partitioned into two parts:
 - I_cache (Instruction cache): instruction cache
 - D_cache (Data cache): data cache.
- Higher-level caches are not partitioned:
 - Partitioning the L1 cache offers the greatest benefit because it is closest to the CPU. The CPU reads/writes directly to the L1 cache. L1 cache needs to support multiple concurrent access commands and optimization measures.
 - Dividing caches at higher levels is not highly efficient and complicates the cache control system.

❖ Methods to reduce misses:

- Good caching:
 - High hit rate
 - Low miss rate
 - Miss penalty is not too slow
- Types of misses:
 - Compulsory misses: usually occur at the time the program is launched, when program code is being loaded into memory and has not yet been loaded into the cache
 - Capacity misses: due to limited cache size, especially in multitasking environments. When the cache is small, program code is frequently switched between cache and memory
 - Conflict misses: due to conflicts when multiple memory lines compete for the same cache line

❖ Methods to reduce misses:

■ Increase cache line size:

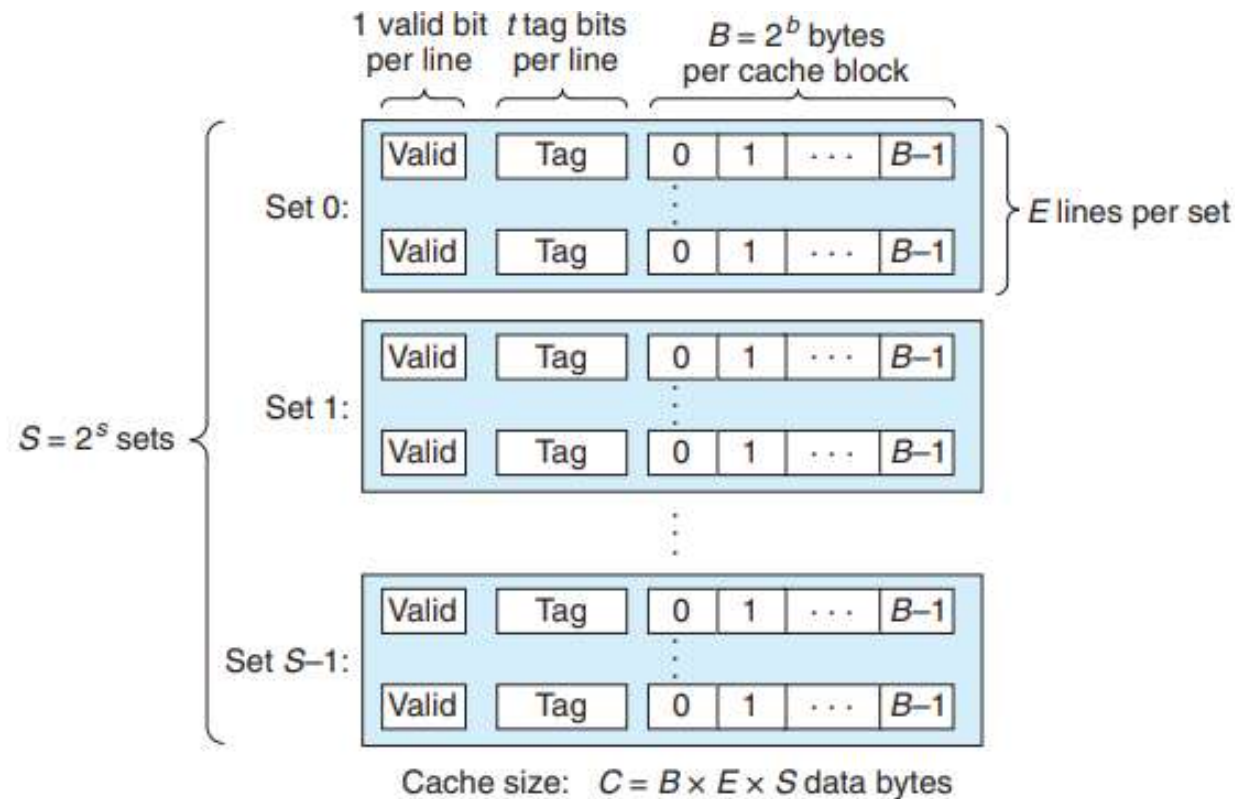
- Reduce compulsory misses: Larger lines have better neighboring coverage
- Increase conflict misses: Larger lines reduce the number of lines in the cache, leading to more memory lines referencing the same cache line, thus increasing conflict misses
- Larger lines can waste cache space
- Commonly used line size is 64 bytes

❖ Methods to reduce misses:

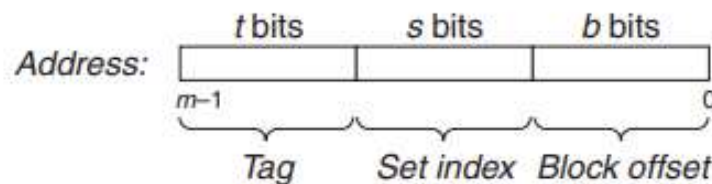
- Increase the degree of associativity (increase the number of cache ways):
 - Reduce conflict misses: Increasing the number of ways allows for more flexible memory-cache mapping or more choices, thus reducing conflict misses
 - Slow down the cache: Increasing the number of ways creates a larger search space, slowing down the cache

- ❖ Calculating the number of bits in the cache:
 1. Calculate the total number of bits required for a direct-mapped cache memory of size 64 KByte and a cache line of size 1 memory word, assuming a 32-bit address bus?
 2. What is the total number of bits needed for a cache memory using a 4-way associative set of size 64 KByte and a cache line of size 1 memory word, assuming a 32-bit address bus?
 3. Calculate the total number of bits required for a direct-mapped cache memory of size 64 KB and a cache line of size 8 memory words, assuming a 32-bit address bus?

❖ Calculating the number of bits in the cache:



(a)



- ❖ Exercise 1: Given a machine with 64KB of main memory, an 8-byte cache, and a cache consisting of 32 lines organized by direct mapping:
1. Determine the number of bits in the address components of a memory cell.
 2. Which cache line is the memory cell with address 0D20H in main memory loaded into?

- ❖ Exercise 2: Assume a 32-bit computer architecture.
Calculate the size of a cache that can hold 64 Kbytes of data, with each block of memory containing 8 words of data, using the following mapping methods:
1. Direct mapping
 2. 4-way set-associative mapping
 3. Compare the two methods above?

- ❖ Exercise 3: Assume that the computer has a 128KiB cache organized using a 4-way set-associative mapping. The cache has a total of 1024 Sets from S0 to S1023. Main memory addresses are 32 bits and each byte is addressed.
1. Calculate the number of bits for the address fields when accessing the cache?
 2. Determine which Set of the cache the memory byte with address D202023A(H) is mapped to?

- ❖ Exercise 4: Computers use 32-bit addressing to address memory in bytes; the data bus connecting to main memory is 32 bits. Please answer the following:
1. What is the maximum number of bytes that can be addressed? What are the start and end addresses in hexadecimal format?
 2. Indicate which memory line the following bytes with addresses 0FE12C3D(H) and 10ABCD06(H) are located on?

- ❖ Exercise 5: Given a computer with 64 Kbytes of byte-addressed main memory and a cache consisting of 32 lines organized as direct mappings, each line measuring 8 bytes.
1. Determine the number of bits in the address fields: Tag, Line, Word.
 2. Indicate which cache line each main memory byte with the address below is loaded into:

0001 0001 0001 1011
1100 0011 0011 0100
1101 0000 1101 1101
1010 1010 1010 1010
 3. Assuming the memory byte with address 0001 1010 0001 1010 is loaded into the cache, indicate the binary addresses of the other memory bytes loaded with that byte on the same line.